



第 22 届湖南省大学生计算机程序设计竞赛
——应用开发类竞赛（2026）

系 统 设 计 说 明 书

作品名称:	码安管家——LLM 驱动的 漏洞分析站
作品类别:	信息安全类
作 者:	白明耀、易卓玥、谭依晴
指导老师:	陈纪友
单 位:	衡阳师范学院

二零二六 年 九 月 二十五 日

目录

前置页 术语与阅读指引	7
一、阅读指引	7
二、术语与缩略语	7
第一篇 项目概述	10
1.1 作品名称与定位	10
1.2 开发背景与问题	10
1.2.1 软件安全形势	10
1.2.2 现有技术路径的局限	11
1.2.3 本作品的解法	11
1.3 目标用户与使用场景	12
1.4 术语与缩略语	12
第二篇 功能需求说明书	13
2.1 功能总览	13
2.2 三大核心任务	14
2.2.1 任务 A：扫描一段代码或一个文件	14
2.2.2 任务 B：批量扫描一个项目并导出报告	15
2.2.3 任务 C：扫描 GitHub 仓库或网页	15
2.3 扫描结果与判定语义	15
2.3.1 四种判定	15
2.3.2 每条结果包含的字段	15
2.3.3 安全评分	16
2.3.4 与传统工具结果差异的说明	16
2.4 Web 界面功能需求	16
2.4.1 四种扫描输入方式	16
2.4.2 高级选项	16
2.4.3 五个页面	17
2.5 编辑器插件功能需求	17
2.6 命令行功能需求	17
2.7 模型管理与离线能力	17
2.8 非功能需求	18
2.8.1 输入限制与支持范围	18
2.8.2 性能预期	18
2.8.3 兼容性需求	18
2.8.4 隐私与安全需求	19
2.8.5 已知能力边界（如实声明）	19
第三篇 概要设计说明书	20
3.1 总体架构	20
3.2 模块划分	21
3.2.1 核心包 graduation_project/（扫描引擎）	21

3.2.2 后端服务 app/backend/	22
3.2.3 启动器 app/launcher/	22
3.2.4 编辑器插件	23
3.2.5 模块依赖约束	23
3.2.6 代码规模（实测口径）	23
3.3 扫描引擎主线	23
3.3.1 Stage 1：工具召回	23
3.3.2 Stage 2：LLM 封闭裁决	24
3.3.3 2.5 代信任层	24
3.3.4 复核兜底	24
3.4 接口设计	24
3.4.1 REST 接口（28 条，实测 2026-09-19）	24
3.4.2 流式通信协议	25
3.4.3 结果交换格式	25
3.5 部署形态与运行环境	26
3.5.1 四种推理后端	26
3.5.2 后端自动解析顺序	26
3.5.3 平台支持矩阵	26
3.5.4 跨平台启动	27
3.6 效果指标（选摘）	27
3.6.0 评测口径与一键复现	27
3.6.1 系统级成绩（两阶段架构）	28
3.6.2 模型演进主线（SFT，87 合成集统一打分）	29
3.6.3 传统工具对比（14 段典型样本）	31
3.6.4 训练成本与曲线	32
第四篇 详细设计说明书	34
4.1 Stage 1 召回设计	34
4.1.1 自研工具 1：TaintTracker（污点分析）	34
4.1.2 自研工具 2：Prefilter（正则预筛）	35
4.1.3 自研工具 3：CodeSlicer（AST 代码切片）	36
4.1.4 自研工具 8：ExternalScanner（外部工具封装层）	36
4.2 Stage 2 裁决设计	37
4.2.1 自一致率置信度	37
4.2.2 裁决档位四分类	37
4.2.3 裁决上下文的行号精确对位	38
4.2.4 训练对齐裁决格式	38
4.3 2.5 代信任层	39
4.3.1 确定性证据门（零 LLM 成本静态核验）	39
4.3.2 Layer 1：共形预测（统计门控）	40
4.3.3 Layer 2：反事实验证（因果门控）	41

4.3.4 信任分级回填（工具-模型自适应闭环）	41
4.3.5 防锚定的三层结构	43
4.3.6 多数票类型投票（回声票加权）	43
4.4 复核兜底机制	43
4.4.1 无候选复核的三档组态	44
4.4.2 无候选复核的判定门槛	45
4.4.3 targeted 组态的三档定向复核	45
4.5 关键工程机制	46
4.5.1 优先级调度器	46
4.5.2 约束解码兜底	46
4.5.3 手写括号深度匹配 JSON 解析	47
4.5.4 CWE 编号确定性纠正	47
4.5.5 行号内容锚定纠正	47
4.5.6 修复建议验证	47
4.5.7 SARIF 2.1.0 标准化导出	48
4.5.8 多模型投票聚合规则	48
4.5.9 模型管理能力门控（501 而非 500）	48
4.5.10 多后端抽象与精度权衡	48
4.5.11 训练/推理一致性保证	49
4.5.12 URL 抓取的多层防护	49
4.5.12.1 库文件与压缩产物的过滤策略	49
4.5.13 GitHub 仓库扫描的沙箱约束	50
4.5.14 向量知识库设计	51
4.5.15 输入限额的前后端双端一致防线	51
4.5.16 国内镜像下载的工程闭环	51
4.5.16.1 进程内推理的墙钟超时（可用性底线）	51
4.5.17 工具层盲区定位（行级提醒，不产生 finding）	52
4.5.18 文件级风险打分与注意力预算调度	52
4.5.19 列表形式命令调用的安全性判定（唯一入口）	53
4.6 设计依据	53
4.6.1 路径依赖与部署设计	53
4.6.2 llama-cpp-python 的两个真实坑	53
4.6.3 依赖冲突定位与固定	53
4.6.4 “存在但损坏”的坏依赖检测	54
4.6.5 CI 的“永不阻断 PR”设计哲学	54
4.6.6 已识别取舍与设计边界（如实声明）	54
4.6.7 设计沿革（可选章节）	55
第五篇 数据存储设计	56
5.1 总体说明	56
5.2 文件存储	56

5.2.1 数据目录总览	56
5.2.2 信号注册表 signal_registry.json	57
5.2.3 共形校准文件与其当前状态	57
5.2.4 规则文件 models/semgrep_rules/	58
5.2.5 模型权重与适配器	58
5.2.6 Schema 与训练口径一致性	58
5.3 向量知识库	59
5.3.1 存储形态	59
5.3.2 设计依据	59
5.3.3 构建与重建	59
5.4 前端持久化	60
5.4.1 localStorage 用途	60
5.4.2 设计要点	60
5.5 数据生命周期	60
5.5.1 运行期数据产生与销毁	60
5.5.2 网络访问边界	60
5.5.3 卸载清理范围	61
5.5.4 数据可信度保障机制	61
第六篇 软件界面设计书	62
6.1 设计系统	62
6.1.1 设计哲学	62
6.1.2 双层令牌体系	62
6.1.3 零依赖手写 SVG 图表	62
6.1.4 视觉规范摘要	63
6.2 信息架构与页面清单	63
6.2.1 页面清单	63
6.2.2 导航结构	63
6.2.3 共享资源	63
6.3 界面截图	64
6.3.1 Web 界面	64
6.3.2 编辑器插件	69
6.3.3 设置与模型管理	70
6.4 关键交互	71
6.4.1 主题切换	71
6.4.2 扫描结果呈现流程	71
6.4.3 交互健壮性设计	72
6.4.4 可访问性	72
6.4.5 编辑器插件交互	72
第七篇 用户操作手册	73
7.1 三个最常用的任务（5 分钟上手）	73

7.1.1 任务 A：扫描一段代码或一个文件	73
7.1.2 任务 B：批量扫描一个项目并导出报告	74
7.1.3 任务 C：扫描 GitHub 仓库或网页	74
7.2 读懂扫描结果	74
7.2.1 四种判定，各有含义	74
7.2.2 每条结果包含什么	75
7.2.3 安全评分	75
7.2.4 为什么结果可能和 Bandit / Semgrep 不一样	75
7.3 Web 界面导览	76
7.3.1 扫描工作台的四种输入方式	77
7.3.2 高级选项	77
7.3.3 API 文档	77
7.4 输入限制与支持范围	78
7.5 高级功能	78
7.6 在编辑器里用（VS Code / IntelliJ 插件）	78
7.6.1 VS Code	78
7.6.2 IntelliJ IDEA	79
7.7 命令行速查	79
7.8 离线使用与模型管理	79
7.9 常见问题 FAQ	81
7.10 已知边界（使用前请读一遍）	81
第八篇 安装手册	82
8.1 系统要求	82
8.2 一键启动（推荐）	82
8.2.1 启动脚本自动完成的步骤	82
8.2.2 ⚠️ Linux 用户：先停用系统级 Ollama 服务（只需一次）	83
8.3 手动启动	84
8.4 确认安装成功	84
8.5 切换推理后端	84
8.5.1 后端平台支持矩阵	84
8.5.2 显存与平台说明	85
8.6 环境变量配置	85
8.7 卸载	86
8.7.1 一键卸载	86
8.7.2 ⚠️ 必须用安装时的同一个 Python 环境卸载	86
8.7.3 卸载范围	86
8.7.4 不会自动删除的内容	87
8.7.5 注意事项	87
8.8 故障排查	87
8.8.1 Semgrep 规则本地化（离线可用，随软件分发）	87

8.8.2 启动相关	87
8.8.3 模型相关	88
8.8.4 RAG / 向量库相关	88
8.8.5 插件相关	89
8.8.6 IDE 报红（不影响运行）	89
8.9 目录结构	89
附录	90
附录 A 环境要求表	90
附录 B CLI 命令速查	90
附录 C 输入限制与支持范围	91
附录 D 评估口径说明	91
D.1 指标定义	91
D.2 口径须知	91
D.3 测试面构成	92
D.4 术语口径提醒	92

前置页 术语与阅读指引

一、阅读指引

本说明书按竞赛材料要求分为八篇，对应关系如下：

官方材料	对应篇章
—（引言性概述）	第一篇 项目概述
功能需求说明书	第二篇 功能需求说明书
概要设计说明书	第三篇 概要设计说明书
详细设计说明书	第四篇 详细设计说明书
数据库设计说明书（如必要）	第五篇 数据存储设计
软件界面设计书	第六篇 软件界面设计书
用户操作手册	第七篇 用户操作手册
安装手册	第八篇 安装手册
—（补充材料）	附录 A~D

二、术语与缩略语

缩略语 / 术语	全称	说明
CWE	Common Weakness Enumeration, 通用弱点枚举	软件安全弱点的标准分类体系，如 CWE-89 SQL 注入
CVE	Common Vulnerabilities and Exposures, 通用漏洞披露	针对具体软件产品的已公开漏洞编号
SAST	Static Application Security Testing, 静态应用安全测试	不运行程序、直接分析源码的安全检测
SCA	Software Composition Analysis, 软件成分分析	检测第三方依赖中的已知漏洞
IaC	Infrastructure as Code, 基础设施即代码	以配置/模板描述基础设施；其配置错误也是安全风险
污点分析	Taint Analysis	跟踪外部输入（source）流向危险操作（sink）的数据流分析技术
Source / Sink	污点来源 / 触发点	漏洞数据流的起点与终点
LLM	Large Language Model, 大语言模型	本系统的语义裁决引擎

缩略语 / 术语	全称	说明
SFT	Supervised Fine-Tuning，监督微调	用标注数据对基座模型做指令微调
LoRA	Low-Rank Adaptation，低秩适配	冻结基座、只训练低秩增量矩阵的参数高效微调方法
QLoRA	Quantized LoRA	4bit 量化基座 + LoRA 微调，显著降低显存需求
NF4	4-bit NormalFloat	一种 4bit 量化数据格式
量化	Quantization	用更低精度表示模型权重以减小体积、加速推理
两阶段扫描	Two-Stage Scanning	本系统核心架构：Stage 1 工具召回 + Stage 2 LLM 裁决
自一致率	Self-Consistency	对同一判定采样多次，以判真票占比作为置信度代理
共形预测	Conformal Prediction	一种带覆盖率保证的统计预测框架，用于门控不确定样本
反事实验证	Counterfactual Verification	在 sink 行内注入防御后重判，以区分“真理解”与“模式匹配”
证据门	Evidence Gate	基于静态特征（已知防御、输入入口）的零 LLM 成本确定性校验
信任分级回填	Trust-Graded Feedback	按判定质量分级，仅高质量判定允许回填工具记忆
SARIF	Static Analysis Results Interchange Format	静态分析结果的标准化交换格式（本系统导出 2.1.0）
NDJSON	Newline-Delimited JSON	每行一个 JSON 对象的流式传输格式
RAG	Retrieval-Augmented Generation，检索增强生成	检索知识库内容注入模型上下文
SSRF	Server-Side Request Forgery，服务端请求伪造	本系统 URL 抓取功能需防御的攻击类型
FPR	False Positive Rate，误报率	安全样本被误判为漏洞的比例
strict / loose 口径	严格 / 宽松评估口径	strict 同时校验判断与 CWE 编号是否正确；loose 只看是否判出漏洞

缩略语 / 术语	全称	说明
未决率	—	转人工复核、不做自动结论的样本占比
Ollama	—	本地大模型运行与模型分发工具
GGUF	GPT-Generated Unified Format	llama.cpp 生态的模型文件格式
vLLM	—	高吞吐大模型推理服务框架
v9max	—	本系统发布会话的模型版本 (Ollama 形态)
$\alpha 0.5$	Nivis-alpha0.5	本系统高精度 LoRA 适配器版本 (Transformers 形态)
Nivis	—	本系统内核引擎与模型系列的代号 (Nivis / Nivis- $\alpha 0.5$)
固定组态 fixed5	—	评估基线组态 (关闭信号回填), 代表“静态管线”能力
wave8	—	第八波收口组态 (盲区收口 + 归因链修正 + 回填开启 + 上下文 16384), 当前最强
未复核 / not_covered	—	工具零召回且预算未覆盖, 模型从未被调用; 区别于“已判安全”
复核覆盖度	review coverage	与判定正交: llm_checked / not_covered / unknown, 表达“模型有没有看过”
注意力预算	Attention Budget	按文件风险分与盲区命中排序, 决定哪些文件值得消耗 LLM 预算
盲区提醒	Blind Spot Hint	以行级中性提示把“工具写不了规则”风险注入裁决 prompt; 不产生候选、不进回填
工具-模型自适应闭环	—	模型裁决经门控校验后回填工具记忆, 两者互相修正 (§4.3.4)

第一篇 项目概述

本篇对应官方材料：引言性概述（置于功能需求说明书之前，提供背景与定位）

1.1 作品名称与定位

作品名称：码安管家——LLM 驱动的漏洞分析站 软件名称：凿凿（ZaoZao）（内核引擎代号 Nivis）

凿凿是一套在用户本机运行的代码安全分析系统。其基本工作方式是：由传统安全工具快速定位代码中的可疑点，再由本地部署的大语言模型对每个可疑点做语义判断——它是不是真漏洞、属于哪一类 CWE、应当如何修复。

系统面向的核心问题，是传统静态分析工具与直接使用大模型这两条路径各自的局限。二者的对比见下表：

维度	传统工具（Bandit/Semgrep）	凿凿（LLM 驱动）
检测方式	固定规则模式匹配	代码语义理解、上下文感知
漏洞覆盖	已知漏洞模式	可发现变体/非典型漏洞
输出形式	漏洞类型 + 规则编号	自然语言解释 + 修复建议
多语言	工具专属规则集	跨语言统一理解
误报控制	规则泛化能力差	上下文判断过滤/净化逻辑

系统定位：它比规则工具“懂语义”（能理解数据流、识别有效防御、解释漏洞原理），比云服务“更私密”（代码、模型、知识库全部留在本机，运行期可完全离线）。

输出内容：对源代码进行安全审计，输出漏洞判定 + CWE 类型 + 风险等级 + 污点来源/触发点 + 自然语言解释 + 修复建议。

1.2 开发背景与问题

1.2.1 软件安全形势

软件安全漏洞造成的经济损失与社会影响持续扩大。IBM《2024 年数据泄露成本报告》显示，全球单次数据泄露事件平均损失达到 488 万美元，较 2023 年增长 10%，为历年最大增幅；其中由系统漏洞利用引发的泄露事件占比持续位于前列。中国国家互联网应急中心（CNCERT）发布的《2024 年上半年我国互联网安全态势报告》指出，上半年收录的通用软硬件漏洞数量同比增长 17.9%，Web 应用层漏洞仍为攻击者主要入口。

在漏洞数量持续增长的同时，安全缺陷的修复成本随发现阶段后移而急剧上升，代码安全审计因此在软件开发生命周期中的位置不断前移，成为 DevSecOps 实践的关键环节。然而人工代码审计受制于审计员的产能——逐条核对规则文档、理解数据流上下文、判断防御是否有效，单条告警的理解与验证往往需要数十分钟量级，难以匹配漏洞增长的速度。

1.2.2 现有技术路径的局限

路径	局限
传统静态分析（SAST）	规则驱动，只能覆盖已知漏洞形态；规则泛化能力差导致高误报；对需要理解数据流与业务语义的漏洞（认证缺失、信任边界绕过）天然覆盖不到
云端大模型直接分析	代码需上传第三方，隐私合规风险；无法离线；输出不稳定，缺可核验的证据链
本地大模型直接全文分析	开放生成任务下模型注意力被长文件稀释，漏报与幻觉并存；无工具证据锚定，结论不可追溯

1.2.3 本作品的解法

针对上述三处局限，本项目做了三个第一性选择——它们共同构成本作品与“把大模型接进扫描器”的分界线：

选择一：把大模型的任务从“发现”改成“判断”。开放生成任务里，模型既要在一整份文件里找线索、又要判断线索真伪，注意力被长文件稀释，结论无法锚定。本系统改为两阶段：Stage 1 由确定性工具（Bandit / Semgrep / Gitleaks / Trivy 等）把可疑点召回齐全——这一步是规则擅长的；Stage 2 由本地大模型对每一条具体候选只回答一个封闭问题：“这条告警是不是真漏洞、属于哪一类 CWE、怎么修”。模型不再需要在全文里“大海捞针”，只需要在带行号的证据片段上做判别。错误的来源因此变得可归因：要么“工具没召回”，要么“裁决判错”，二者可分开度量、分别修复。

选择二：让工具与模型互相修正，而不是单向调用。裁决结论不直接采信、也不直接丢弃，而是先经信任层分级，只有通过五重门控的高质量判定才被允许回填工具层记忆（候选排序加权 / 类型映射校正 / 按文件的负向记忆降权）。这意味着工具不只是“给模型喂线索”，它自己的误报特征也在被模型的判定修正——形成工具-模型自适应闭环。为防止闭环反过来污染确定性工具，回填门槛被写成代码参数而非口头原则。

选择三：如实表达“不确定”和“没看过”。安全工具最常见的失效模式不是判错，而是把“没检查”呈现成“没问题”。本系统把结论拆成四态：发现漏洞 / 安全 / 需人工复核 / 未复核。低置信判定转人工复核；受限于注意力预算而未被模型处理的文件标注“未复核”，不冒充“已判安全”。所有门控在这条原则上是一致的——它们只负责把判定降级到人工复核，从不代替人下结论。

此外，三项工程约束贯穿始终：完全离线（代码、模型、知识库均不出本机）、可复现（评估口径与冻结测试集固定，一键复现）、可卸载（全部数据在项目目录内，分级清理）。

架构与门控细节见第三篇、第四篇；产品形态与操作方式见第六至八篇。

1.3 目标用户与使用场景

目标用户	典型场景	使用方式
开发者个人	交代码前自查一段“拿不准安全不安全”的逻辑	Web 界面粘贴代码
开发团队 / 项目负责人	提交 / 发布前把整个项目过一遍	批量扫描 + 导出 Markdown 报告
采购 / 引入方	评估一个开源仓库能不能引入	GitHub 仓库扫描
日常编码者	写代码时随手查当前文件	VS Code / IntelliJ 插件
自动化流水线	提交前检查、CI 集成	命令行工具 CLI

系统提供四种使用方式：Web 界面、命令行工具、VS Code 插件、IntelliJ 插件。四者共享同一个后端服务，功能对等。

1.4 术语与缩略语

完整术语表见前置页《术语与阅读指引》，此处不重复。

本篇及后续篇章高频术语速览：

术语	一句话说明
CWE / CVE	弱点分类编号 / 具体漏洞编号
污点分析	跟踪外部输入流向危险操作的数据流分析
两阶段扫描	工具召回 + LLM 裁决的架构
自一致率	多次采样的判真票占比，作为置信度代理
共形预测 / 反事实验证 / 证据门	2.5 代信任层的三道门控
v9max / $\alpha 0.5$	系统的两种模型发布形态（Ollama / Transformers+LoRA）

第二篇 功能需求说明书

本篇对应官方材料：功能需求说明书

2.1 功能总览

本节给出系统全部功能点的清单，按功能域拆解，供按官方清单逐项勾对。

功能域	功能点	说明	主要载体
代码扫描	粘贴代码扫描	选择语言后粘贴源码，单文件分析	Web 扫描工作台
	文件上传扫描	拖拽/选择多文件，逐个返回结果	Web / CLI
	URL 扫描	抓取网页内 <code><script></code> 内容 逐个扫描	Web / CLI
	GitHub 仓库扫描	浅克隆仓库 → 遍历代码文件 → 批量扫描	Web / CLI
	批量目录扫描	递归扫描目录下所有支持的文件	CLI
结果呈现	漏洞判定（四态）	发现漏洞 / 安全 / 需人工复核 / 未复核（未送 LLM，不冒充「已判安全」）	全平台
	来源筛选	按任务类型（粘贴 / 上传 / URL / 仓库 / 演示样本 / 多模型）过滤结果，与严重度筛选叠加；无该类结果时对应按钮自动隐藏	Web
	CWE 类型与风险等级	标准编号 + Critical/High/Medium/Low	全平台
	Source & Sink 定位	污点来源行与触发行，可直达代码行	Web / 插件
	漏洞说明与修复建议	自然语言解释 + 行号锚定的局部修复建议	全平台

功能域	功能点	说明	主要载体
	信任层明细	裁决档位、一致性置信度、共形预测、反事实、证据门	Web / 插件
	安全评分	按风险等级累计扣分的评分圆环	Web
报告导出	Markdown 报告	单文件 / 批量报告下载	Web
	SARIF 2.1.0 导出	与 GitHub Code Scanning 等互通	API
编辑器集成	VS Code 插件	单文件扫描 + 诊断波浪线 + 批量工作区扫描	VS Code
	IntelliJ 插件	选区扫描 + 气球通知	IntelliJ
命令行	health / scan / batch / url / github	无需启动后端，直接复用扫描引擎	CLI
模型管理	模型切换 / 拉取 / 删除	带流式下载进度	Web / CLI
	多后端选择	Ollama / Transformers / LlamaCPP / vLLM	环境变量 / 启动器
	多模型投票	选 ≥ 2 个模型分别扫描再投票聚合	Web
知识增强	RAG 知识库	检索内置漏洞知识注入判定过程	可选开关
	CWE 样本库	25 类常见漏洞的可搜索示例与修复方案	Web
运维	健康检查	分层探针（存活/深度）	API / CLI
	扫描队列	优先级调度（交互式优先于批量）	API
	一键启动 / 卸载	跨平台脚本，依赖自动安装	启动器

2.2 三大核心任务

本节规定三大任务的功能需求；逐步操作说明统一见第七篇 §7.1（5 分钟上手）。

2.2.1 任务 A：扫描一段代码或一个文件

用户在扫描工作台粘贴代码或上传文件后发起分析，结果区给出：是否漏洞、CWE 类型、风险等级、漏洞原理说明与修复建议；单文件典型耗时 5~15 秒，长文件更久。

2.2.2 任务 B：批量扫描一个项目并导出报告

支持一次拖入整个模块的代码文件（最多 200 个），批量扫描时实时显示逐文件进度；完成后可导出 Markdown 报告（漏洞汇总表 + 每个文件的详细结果 + 修复建议）。

2.2.3 任务 C：扫描 GitHub 仓库或网页

支持 GitHub 仓库输入（浅克隆：只读、120 秒超时、扫完即删，不在用户机器上留下仓库副本）与网页 URL 输入（抓取页面脚本逐个分析）；两者均可限定最大扫描文件数（网页端默认 20）。

2.3 扫描结果与判定语义

2.3.1 四种判定

系统的判定不仅表达“是/不是漏洞”，还如实表达“我有没有看过”。后一件事常被安全工具省略，结果是“没扫到”被读成“没问题”：

判定	含义	用户应当做什么
发现漏洞	系统确信存在漏洞（模型多次采样一致 + 通过证据校验）	按修复建议处理
安全	已实际复核且未发现问题（工具没召回可疑点，且该文件确实进入了复核）	常规流程即可
需人工复核	系统有怀疑但证据不足，故意不替用户下结论	花一分钟看一眼标注位置
未复核	工具零召回、且本轮的注意力预算没有覆盖到该文件——LLM 从未被调用	按需人工查看；不要把它当作“已确认安全”

「需人工复核」不是出错，是设计：本系统的原则是“宁可多一道人工复核，也不漏报真漏洞、不错杀不确定的判定”。复核结果单列，不计入安全评分。

「未复核」同样不是出错：语义分析要花推理时间，系统用一个显式的注意力预算（按文件风险分与工具盲区命中排序）决定哪些文件值得花预算。没被覆盖到的文件如实标注为“未复核”——保留“未复核”这个状态，正是为了不让它在界面上被渲染成“安全”。这是“结论必须可解释”这一原则在结果模型上的落点。

2.3.2 每条结果包含的字段

- **CWE 类型**：漏洞编号与名称（如 CWE-89 SQL 注入）；
- **风险等级**：Critical / High / Medium / Low；
- **Source & Sink**：污点数据的来源行与触发行（有数据流的漏洞才显示），配合编辑器插件可直达代码行；
- **漏洞说明**：自然语言解释“为什么这是漏洞”；

- 修复建议：行号锚定的局部修复建议；
- 信任层明细：每条候选的裁决档位、一致性置信度、CWE 类型、共形预测与反事实验证状态、证据门是否拦截（低置信候选列入“需人工复核”清单）；
- 模型原始输出：可折叠查看，用于核对。

2.3.3 安全评分

系统按 Critical 每项 -8 / High -8 / Medium -4 / Low -2 / Info -0.5 累计扣分给出安全评分；“需人工复核”与“未复核”的样本均单列展示、不参与扣分，保证评分语义与复核信号不混。

2.3.4 与传统工具结果差异的说明

本系统的结果可能与传统工具（Bandit / Semgrep）不同：放行参数化查询、subprocess 列表参数等带有效防御的“安全写法”；报告规则覆盖不到的语义类漏洞；对低置信判定给出“需人工复核”而非随机硬判。三类差异均属正常设计，面向用户的解释见第七篇 §7.2.4。

2.4 Web 界面功能需求

2.4.1 四种扫描输入方式

输入方式	交互	说明
粘贴代码	选语言下拉框 + 文本框	Python / JavaScript / TypeScript / Java / PHP / Go / HTML
上传文件	拖拽或点击选择，支持多文件	批量时显示实时进度条，完成后可下载 Markdown 报告
URL 扫描	输入目标站点 URL	自动抓取页面 <script> 内容逐个扫描
GitHub 仓库	输入仓库地址 + 最大文件数	浅克隆 → 遍历 → 批量扫描

2.4.2 高级选项

选项	说明
多模型投票	勾选后选择 ≥2 个模型，系统顺序加载各模型并投票聚合结果（更准但更慢）
两阶段扫描	系统唯一扫描路径（工具召回 + LLM 自一致率裁决），无需勾选即生效；该开关仅控制下方采样数设置是否可调
自一致率采样数	两阶段 LLM 裁决的采样次数（1~10，默认 3）。采样越多越准、越慢

外部工具（Bandit / Semgrep / Gitleaks / Trivy）已内置于两阶段 Stage 1 工具召回，无需单独勾选：密钥（secret）与依赖漏洞（SCA）类发现由确定性工具直接报告，SAST / IaC 类发现进入 LLM 裁决层判定真伪。

2.4.3 五个页面

页面	路径	功能
欢迎页	/welcome.html	首次访问的引导页（系统定位与入口导航）
仪表盘	/index.html	扫描历史统计、引擎健康状态、最近发现漏洞一览
扫描工作台	/scan.html	核心扫描入口，支持四种输入方式
CWE 样本库	/cwe.html	25 类常见 CWE 漏洞的代码示例与修复方案（可搜索）
安全态势	/posture.html	安全态势总览

2.5 编辑器插件功能需求

功能	VS Code 插件	IntelliJ 插件
单文件扫描	✅ 右键 / 命令面板 / 标题栏图标	✅ 选中代码或整文件
结果显示	Webview 面板（信任层明细表格）	气球通知（信任层摘要）
诊断标记	红色波浪线（按确认裁决逐处标记）	暂未实现
批量扫描	✅ 支持（工作区 / 指定文件夹）	暂未实现
自动扫描	✅ 保存时自动扫描（实验性，1.5s 防抖）	—
后端地址配置	✅ vulnScanner.backendUrl	✅ 弹窗中可改基础地址

前置条件：后端服务已启动并保持运行（插件只是“遥控”后端，后端关闭则插件不能工作）。

2.6 命令行功能需求

命令	功能
health	健康检查（模型/服务是否可用）
scan <file>	扫描单个文件
batch <dir>	批量扫描目录下所有代码文件
url <url>	扫描 URL 抓取的脚本
github <repo>	扫描 GitHub 仓库（浅克隆）

全局参数：--model（切换模型）、--ollama-url、--rag（知识库）、--format text|json、--output。

特点：直接复用扫描引擎，无需启动后端服务；扫描失败时退出码为 1，适合接入 CI 或脚本化批量检查。

2.7 模型管理与离线能力

- 完全离线运行：联网只是首次下载依赖和模型时需要。系统运行期不访问外网（HuggingFace 强制离线模式、Semgrep 规则已随软件本地化）；
- 换 / 加模型：Web 界面右上角模型管理抽屉可拉取、删除、切换模型（带实时下载进度）；
命令行方式：--model <模型名> 或环境变量 VULN_SCANNER_MODEL；

- 两档模型形态（界面会显示当前形态）：
 - 发布模型 v9max（Ollama，约 4.7GB，兼容性最好）；
 - 高精度形态（Transformers + LoRA）——启动器检测到本机环境可运行时自动启用；跑不动会自动回退并打印说明；
- 国内网络：依赖走国内镜像、模型走 HF 镜像，支持断点续传与断流自动换源，一般无需手动配置代理。

2.8 非功能需求

2.8.1 输入限制与支持范围

项目	限制
代码语言	Python / JavaScript / TypeScript / Java / PHP / Go / HTML
文件类型	.py .js .ts .jsx .tsx .java .php .go .html .htm .vue .svelte
单段代码	最多 200 万字符
单文件	最大 2 MB
单次批量	最多 200 个文件、总计 10 MB
GitHub 扫描	浅克隆 + 120 秒超时 + 文件数上限（网页端默认 20，接口默认 50，可调，上限 500）

超限会被前端直接拦截提示（后端再做一道同样的校验，返回 413），不会让用户白等。

2.8.2 性能预期

场景	预期
单文件扫描	典型 5~15 秒；长文件（自动切片后逐段分析）可能到分钟级
批量扫描	逐文件流式返回，页面实时显示进度
启动	首次需下载依赖与模型（取决于网速）；此后通常 10 秒内完成
无 GPU	自动回退 CPU 推理，速度约为 GPU 的 1/10~1/20

2.8.3 兼容性需求

条件	要求
操作系统	Windows 10+ / Linux / macOS
Python	3.10+
磁盘空间	≥ 8 GB（建议 15 GB）
内存	≥ 8 GB（建议 16 GB）
显卡（可选）	NVIDIA / AMD(ROCm) / Apple Silicon ≥ 4 GB；可无 GPU

2.8.4 隐私与安全需求

- 全离线：代码、模型、知识库全部留在本机，运行期不联网；
- 无数据外传：扫描内容不上传任何第三方服务；
- URL 抓取的自我防护：仅允许 http/https 协议，拦截内网/回环/链路本地/保留地址，缓解 DNS rebinding，逐跳校验重定向（最多 5 跳），响应体流式读取并限制大小。

2.8.5 已知能力边界（如实声明）

1. 速度：语义分析的成本是时间——定位是“审计增强”，不是秒级 lint；
2. “需人工复核”会占一定比例：系统对没有确定性证据的判定选择保守（宁可交给人工）。这是防漏报、防误判的设计代价，不是故障；
3. 两类漏洞是弱项：① 跨文件的复杂攻击路径（只扫单文件时看不到全局数据流）；② “缺失型”漏洞（认证缺失、CSRF、竞态等“该有而没有”的问题）——此类没有确定性校验手段，置信不足时都会转人工复核；
4. 两档部署形态精度不同：高精度形态（Transformers + LoRA）优于发布模型（Ollama 量化形态）。界面会如实显示当前用哪一档；
5. 结论是辅助，不是法律：所有“安全”判定都建议结合业务上下文理解——系统能看懂代码语义，但看不懂未写出来的需求约束；
6. “未复核”会按注意力预算出现：大仓库批量扫描时，零召回且无风险特征的文件不会被送入模型。系统如实标注该状态而不是判定为安全——若需要每个文件都过一遍模型，可切换为全量复核组态（见第三篇 §3.1 组态说明）。

第三篇 概要设计说明书

本篇对应官方材料：概要设计说明书

3.1 总体架构

系统采用两阶段扫描架构：Stage 1 用确定性工具把可疑点召回齐全，Stage 2 让本地大模型对每个可疑点做封闭裁决；裁决结果再经 2.5 代信任层门控、按文件聚合为四态结论、对工具盲区做兜底复核。整体数据流如图 3-1 所示：

- **Stage 1 · 工具召回（4 路并行）**：污点规则 / AST 轻量污点 / 正则预筛 / 外部工具四路并行（逐路设计见 §3.3.1）；
- 分流：有候选 → Stage 2 逐条裁决；无候选 → 按注意力预算复核（见 §3.1 组态说明）；
- **Stage 2 · LLM 封闭裁决**：N=3 次独立采样，以自一致率作置信度；
- **2.5 代信任层**：确定性证据门（零成本） / 共形预测（统计） / 反事实验证（因果） / 信任分级回填；
- 聚合 → 兜底：文件级多数票聚合为四态结论；全部候选被否决的文件再做一次全文件兜底复核。

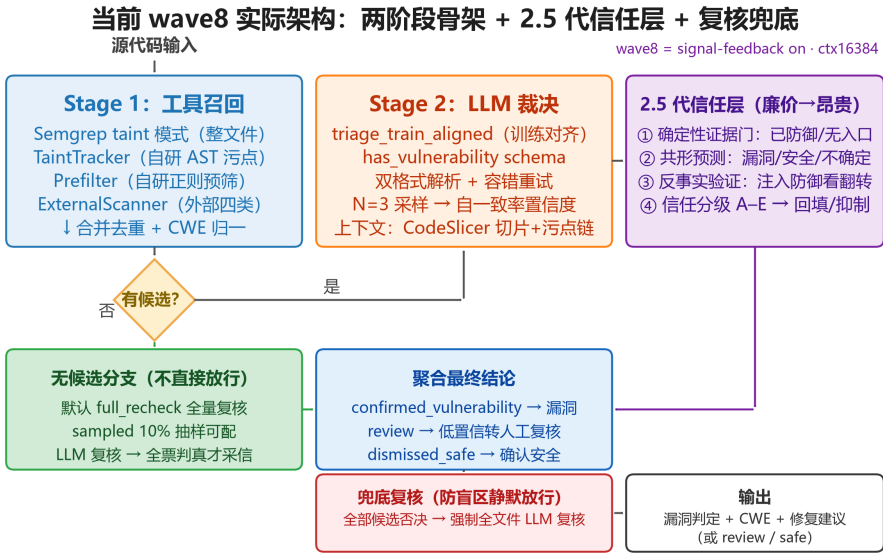


图 3-1 两阶段扫描架构（完整形态：Stage 1 工具召回 → Stage 2 LLM 裁决 → 2.5 代信任层 → 聚合结论 → 兜底复核 → 输出；4 路并行召回 / N=3 裁决 / 按注意力预算的无候选复核）

图注补充：图中「无候选复核」方框对应 no_candidate_mode 的三档组态（targeted 定向复核 / full_recheck 全量复核 / sampled 抽样复核）。生产环境所有批量与交互入口的默认组态是 targeted，三档口径见第四篇 §4.4.1。

架构的核心反转：LLM 的任务从“在全文中发现漏洞”（开放生成）变为“对具体 finding 判定真伪”（封闭判别）。只取包含 source/sink 行的 chunk 作为裁决上下文，聚焦 LLM 注意力；每条判定有工具证据做锚点，幻觉空间被大幅压缩。

关键组态（代码核实）：

项目	值	说明
triage_aligned	True	裁决 prompt 使用训练对齐的 has_vulnerability 格式
no_candidate_mode	全局默认 targeted（三档定向复核）	无候选文件的复核策略；full_recheck 仅用于 vLLM 端点与显式指定的演示/消融场景（见第四篇 §4.4.1）
n_samples	3	裁决采样次数

组态口径：生产环境所有批量与交互入口的一致行为是 targeted 三档定向复核（推理预算按“文件风险分 × 工具盲区命中”集中到真正有攻击面的文件上）；sampled（10% 抽样）为扫描器的类默认与评估组态。三档的完整对照、场景映射与环境变量覆盖方式见第四篇 §4.4.1。

3.2 模块划分

系统自下而上分为五层，共 30+ 个模块（实测于 2026-09-19）：

3.2.1 核心包 graduation_project/（扫描引擎）

模块	职责
two_stage_scanner.py	两阶段扫描器（Stage1 召回 + Stage2 裁决 + 信任层 + 兜底）
scanner.py	单阶段扫描引擎（CLI 复用）
taint_tracker.py	基于 tree-sitter 的轻量污点分析
prefilter.py	正则预过滤层
code_slicer.py	AST 代码切片
external_scanner.py	第三方安全工具封装层
cwe_normalizer.py	CWE 编号确定性纠正
line_normalizer.py	行号内容锚定纠正
fix_verifier.py	修复建议验证
sarif_report.py	SARIF 2.1.0 导出
conformal.py	共形预测

模块	职责
counterfactual.py	反事实验证
signal_registry.py	信任分级回填（信号注册表）
chroma_manager.py	向量知识库管理
blind_spots.py	工具层盲区行级提醒（不产生 finding）
risk_budget.py	文件级风险打分与注意力预算调度
shell_safety.py	列表形式命令调用的安全性判定（唯一入口）
prompts.py / schema.py	提示词与输出契约
transformers_client.py / llamacpp_client.py / ollama_client.py / llm_client.py / vllm_client.py	推理后端客户端
paths.py / result_types.py	路径解析 / 结果数据结构

3.2.2 后端服务 app/backend/

模块	职责
main.py	FastAPI 主应用（28 条 REST 路由）
services/scheduler.py	优先级扫描调度器
services/scanner.py	多后端抽象与精度权衡
services/model_registry.py	模型注册表
services/multi_model_scanner.py	多模型投票扫描
services/fetcher.py	URL 抓取（SSRF 多层防护）
services/reporter.py	Markdown 报告生成
static/	Web 前端 5 页 + 共享资源

3.2.3 启动器 app/launcher/

模块	职责
bootstrap.py	硬件检测 + 自适应配置
dependency_installer.py	跨平台依赖安装
install_llamacpp_gpu.py	llama-cpp-python GPU 编译
vllm_server.py	vLLM 常驻服务封装
vuln_scanner_cli.py	命令行工具
start_windows.bat / start_linux_macos.sh	一键启动脚本

3.2.4 编辑器插件

模块	职责
app/vscode-extension/extension.js	VS Code 插件（诊断标记 + Webview + 批量扫描）
app/intellij-extension/.../VulnScannerAction.java	IntelliJ 插件（气球通知）

3.2.5 模块依赖约束

核心层 graduation_project/ 不得反向依赖 app/ 层。该约束已 CI 化：.github/workflows/pr-review.yml 通过静态检查，若核心层出现对 app 层的反向依赖则阻断提交。

3.2.6 代码规模（实测口径）

截至 2026-09-21，自研代码合计约 4.5 万行（140 个源文件，含空行与注释）：

部分	规模
扫描引擎 graduation_project/	25 个文件 / 约 2.0 万行
后端服务 app/backend/（main.py + 6 个服务模块）	约 7,300 行
Web 前端 static/（5 页面，html/js/css）	13 个文件 / 约 9,700 行
启动器 app/launcher/	6 个文件 / 约 6,300 行
双编辑器插件	约 2,100 行

开发节奏：自 2026-06-28 首次提交起，85 天内完成 277 次提交（git rev-list --count, 2026-09-21 实测；引用时须注明统计日期，该数字随提交增长）。

3.3 扫描引擎主线

3.3.1 Stage 1：工具召回

四路并行召回（线程池 max_workers=4；subprocess 型工具释放 GIL，延迟从“顺序求和”降为“取最大”）：

路径	手段	产出
Semgrep taint	自研规则六件套（taint 四件：sqli/cmdi/xss/codei；结构化补位两件：go_cmdi_shell/c_cmdi_system）	带数据流的候选 + 工具缺口补位
TaintTracker	tree-sitter 线性数据流污点分析	source→sink 路径
Prefilter	正则预筛（漏洞特征/安全特征）	候选 + 置信标记
ExternalScanner	Bandit / Semgrep / Gitleaks / Trivy / pip-audit / detect-secrets	secret / sca 直出，sast / iac 进裁决

跨工具去重：semgrep OSS 的 taint JSON 不含 metavar (source/sink 为空)，与 TaintTracker 命中同一条数据流时主键永不相等。去重逻辑用 (taint_type, sink_line) 辅助索引把“空证据候选”归并到已有 finding，保证同一条流只裁决一次。

3.3.2 Stage 2: LLM 封闭裁决

对每个候选构造裁决 prompt，进行 N=3 次采样 (温度 > 0)，以有效票判真票占比作为置信度：

- 置信度 $\geq 0.8 \rightarrow$ 自动判定；
- $0.5 \sim 0.8 \rightarrow$ 标记“需人工复核”；
- $< 0.5 \rightarrow$ 不自动结论。

分母使用有效票 (votes_true + votes_false) 而非 N——解析失败/推理出错的票不代表“否决”，除以 N 会稀释真实一致率。

3.3.3 2.5 代信任层

在裁决之上叠加三道门控 (确定性证据门 / 共形预测 / 反事实验证) 与五重回填门控的信任分级回填，详见第四篇 §4.3。

3.3.4 复核兜底

系统共有两类复核出口：无候选复核 (内部按 no_candidate_mode 分 full_recheck / targeted / sampled 三档组态) / 裁决全否决兜底复核。详见第四篇 §4.4。

3.4 接口设计

3.4.1 REST 接口 (28 条，实测 2026-09-19)

方法	路径	功能
GET	/	首页入口 (重定向到 Web 界面)
GET	/api/health	深度健康检查
GET	/api/health/live	轻量存活探针 (启动器用)
GET	/api/stats	仪表盘统计
GET	/api/backend/info	后端精度报告 (量化位宽 / LoRA / 设备)
GET	/api/backend/options	可选后端列表
GET	/api/models/installed	已安装模型
GET	/api/models/registry	模型注册表
GET	/api/models/local-resources	本地模型资源
POST	/api/models/pull	拉取模型 (流式进度)
POST	/api/models/download-hf	下载 HuggingFace 模型
POST	/api/models/download-gguf	下载 GGUF 模型

方法	路径	功能
POST	/api/models/activate	切换激活模型
DELETE	/api/models/{model_name:path}	删除模型
POST	/api/analyze	单文件分析（两阶段，默认路径）
POST	/api/analyze/two-stage	两阶段扫描（保留 n_samples 与 SARIF 导出）
POST	/api/batch	批量扫描（NDJSON 流式进度）
POST	/api/url-scan	URL 抓取扫描
POST	/api/github-scan	GitHub 仓库扫描
POST	/api/external-scan	外部工具直出扫描（不经 LLM）
POST	/api/verify-fix	修复建议验证
POST	/api/multi-model-scan	多模型投票扫描
POST	/api/vllm-analyze	vLLM 后端分析
GET	/api/report	批量报告下载（Markdown）
POST	/api/report/single	单文件报告下载
GET	/api/scan/result/{job_id}	按 job 取扫描结果
GET	/api/queue/status	调度队列状态
POST	/api/queue/cancel/{task_id}	取消排队任务

完整交互式文档：访问 <http://localhost:8765/docs>（Swagger UI）。

3.4.2 流式通信协议

场景	协议	说明
批量扫描	NDJSON（每行一个 JSON）	前端逐行解析显示进度
模型下载	NDJSON + 心跳保活	空闲 2s 发 heartbeat 帧；响应头 X-Accel-Buffering: no 防缓冲
单文件分析	普通 JSON	—

3.4.3 结果交换格式

- **SARIF 2.1.0**：两阶段结果按置信度映射 level（ ≥ 0.8 error / $0.5 \sim 0.8$ warning / < 0.5 note）；每条 confirmed finding 携带 confidence / votes / source / sink / 传播链 / 修复建议；带 partialFingerprints 稳定哈希，可跨扫描去重。与 GitHub Code Scanning、VS Code Problems、JetBrains Qodana 原生互通。
- **Markdown 报告**：表头（生成时间/语言/耗时）+ 漏洞字段表 + 说明 + 修复建议 + 模型原始输出折叠块；批量版含漏洞汇总表。

3.5 部署形态与运行环境

3.5.1 四种推理后端

后端	形态	定位
Ollama（默认兜底）	Q4_K_M 合并量化的发布模型 v9max	兼容性最好、依赖最少
Transformers	NF4 基座 + FP16 LoRA 进程内推理	精度最高（保 LoRA FP16）
LlamaCPP	Q4 GGUF 基座 + 运行时 FP16 LoRA	速度与精度折中（实验性）
vLLM	常驻服务进程（AWQ/GPTQ 4bit，--enable-lora 挂载 LoRA）	高吞吐服务化

3.5.2 后端自动解析顺序

- 1. VULN_SCANNER_BACKEND 显式设置时优先（ollama / transformers / llamacpp）；
- 2. 配置了 VULN_SCANNER_ADAPTER，或软件目录下探测到合法 LoRA adapter（优先 α0.5 stage2）且运行时兼容时，自动选 Transformers；
- 3. 否则回退 Ollama——探测到 adapter 但本机跑不动 transformers 时也会自动回退并打印告警。

3.5.3 平台支持矩阵

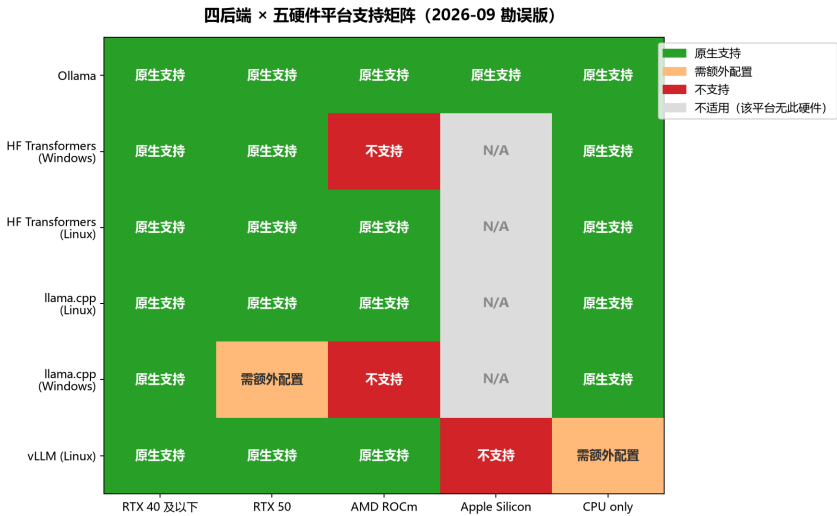


图 3-2 多后端 × 多平台支持矩阵

后端 \ 硬件	RTX 40 及以下	RTX 50	AMD(ROCm)	Apple Silicon	纯 CPU
Ollama (兼容兜底)	●	●	●	●	●
Transformers · Windows	●	●	—	N/A	●
Transformers · Linux	●	●	●	N/A	●
LlamaCPP · Linux	●	●	●	N/A	●
LlamaCPP · Windows	●	● 需额外配置	—	N/A	●
vLLM (仅 Linux)	●	●	●	—	●

● 原生支持；● 需额外配置；— 不支持；N/A 组合不存在（Apple Silicon 为 macOS 专属硬件）。HF 为 Hugging Face Transformers 缩写。

结论：追求设备兼容性优先选 Ollama 后端（全平台全硬件兜底）；Linux 端四种后端支持性普遍最好，Windows 端在 ROCm 与 vLLM 上不可用、RTX 50 系需按上表分别处理。

3.5.4 跨平台启动

一键启动脚本自动完成：依赖检测与安装 → 推理后端选择 → 后端专属依赖安装 → 硬件检测与自适应配置（显存分档选择 num_ctx / num_gpu / 量化等级）→ 模型拉取 / adapter 定位 → 传统安全工具召回安装 → 启动 FastAPI 服务（127.0.0.1:8765）→ 打开浏览器。

3.6 效果指标（选摘）

3.6.0 评测口径与一键复现

本系统的全部成绩数字来自一套冻结且可复现的评测体系，构成如下：

要素	内容
测试面	87 段合成集 + 20 段真实 CVE-fix 片段 + 50 段真实完整文件（rolling_dev，哈希冻结）
官方答案	2026-09-02 冻结（此后测试集不再变动）
风险权重表	experiments/scoring_config/weights_20260903.json（风险加权分依据）
复现命令	python scripts/score_batch.py（一键对指定结果文件按统一口径打分）

三个计分口径互不重叠：strict = 判真且 CWE 归因正确（最严）；acc = 已裁决样本上的判断正确率（不含未决）；未决率 = 转人工复核的占比（与 acc 成对引用，未决样本不计入 acc 分母）。所有口径定义详见附录 D。

数字口径：本节数字为项目统一评估口径（2026-09-08），均可由随附评估脚本一键复现；口径定义见附录 D。

3.6.1 系统级成绩（两阶段架构）

系统组态	测试集	recall	FPR	acc	未决率	strict	加权
两阶段·完整信任层	87 合成	1.000	0.0435	0.987	13.8%	0.923	0.862
两阶段·干净评估	87 合成	1.000	0.0435	0.987	12.6%	0.774	0.786
两阶段·真实 CVE 片段	CVE-fix 20	0.941	—	0.941	15.0%	1.000	0.864
两阶段·真实完整文件	rolling_dev 50	0.645	—	0.645	38.0%	0.450	0.480
纯 LLM 对照	87 合成	0.967	0.154	0.931	0	0.898	0.829

架构收益直观对比（两阶段 vs 纯 LLM）：FPR 由 0.154 收敛至 0.0435；CWE 归因 strict 由 0.898 提升至 0.923。

架构级消融视角：上表本质是一组组态消融——“纯 LLM 对照”= 去掉 Stage 1 工具召回与信任层；“两阶段·干净评估”（fixed5）= 完整管线关闭信号回填；“两阶段·完整信任层”（wave8）= 回填开启。三列之差即各层贡献。逐层细粒度消融的开关已在代码中固化（two_stage_scanner.py 构造参数：use_conformal / use_counterfactual / use_signal_feedback / use_blind_spots，以及四路召回开关 use_semgrep / use_taint_tracker / use_prefilter / use_external 与 no_candidate_mode 三档），评估脚本可逐层关闭；本说明书只引用已实测组态的数字，逐层开关的细粒度数字不在此虚列。

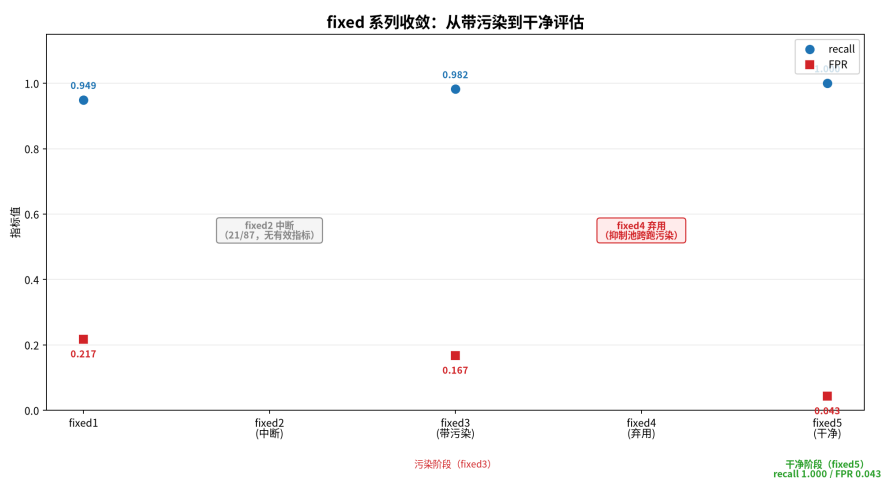


图 3-3 fixed 系列收敛阶梯 (fixed1 0.949/0.217 → fixed3 0.982/0.167 带污染 → fixed5 1.000/0.043 干净；标注“污染教训 → 隔离修复”三段)

该图直观呈现“评估环境隔离”带来的 FPR 收敛：带污染的 fixed3 曾使 FPR 停在 0.167，定位污染源（工具记忆跨轮次累积 / in-sample 校准 / CWE 口径泄漏）并隔离后，fixed5 收敛至 0.043。

测试集梯度：合成 87 段（典型性 72%）→ 真实 CVE 片段 20 段（60%）→ 真实完整文件 50 段（6%），真实度上升而成绩下降的梯度本身即为发现——合成形态的高召回不可直接外推。

组态说明（重要）：上表数字是评估组态下的成绩（关闭信号回填、禁用共形校准自动加载；sampled 抽样复核），生产默认为 targeted 定向复核（见 §3.1）——两者在“未决率”与“单文件复核成本”上有差异，引用数字时须注明组态。

3.6.2 模型演进主线（SFT，87 合成集统一打分）

版本	recall	FPR	strict	真实 CVE recall (测试面)
基线 qwen3-8b 零样本	0.967	0.269	0.848	0.375 (8 段小集)
v5 (首个可信基线)	1.000	0.231	0.836	0.571 (7 段小集)
v9max	1.000	0.423	0.754	0.950 (20 段)
α0.5-combined (纯 LLM 主数据)	0.967	0.154	0.898	0.850 (20 段, no-merge)

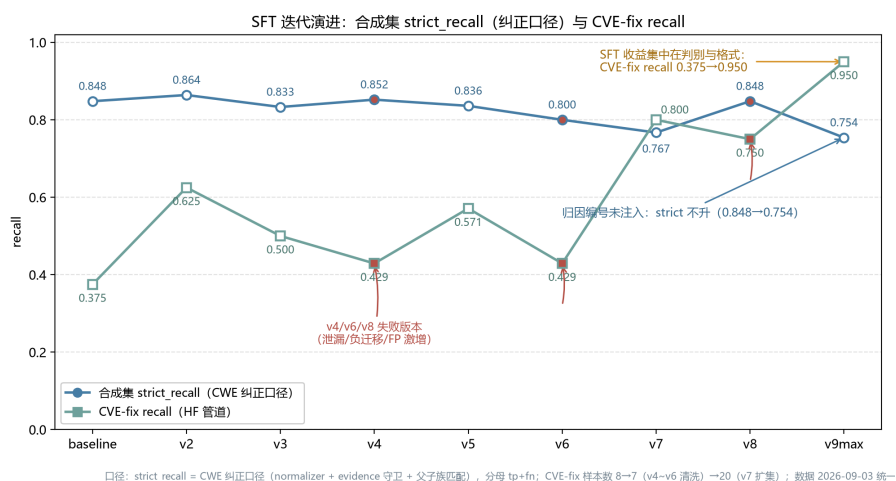


图 3-4 SFT 演进曲线（合成集 strict_recall 与真实 CVE recall 双曲线，红色短箭头标注 v4 / v6 / v8 三个失败版本）

⚠ 口径警示：表中「真实 CVE recall」列前后两行测试面不同——基线（0.375）与 v5（0.571）来自 8 段 / 7 段小集，v9max 与 $\alpha 0.5$ 来自 20 段扩展集。两段分布不可直接比较（扩展集含较多手工样本会拉高数值）；引述时须带测试面标注。

核心结论：SFT 的收益集中在判别与格式（真实 CVE recall 在同口径 20 段上达 0.950）；CWE 归因不随 SFT 提升，靠后处理纠正 + 两阶段架构补足；FPR 的最终收敛靠两阶段信任层（0.154→0.0435）。

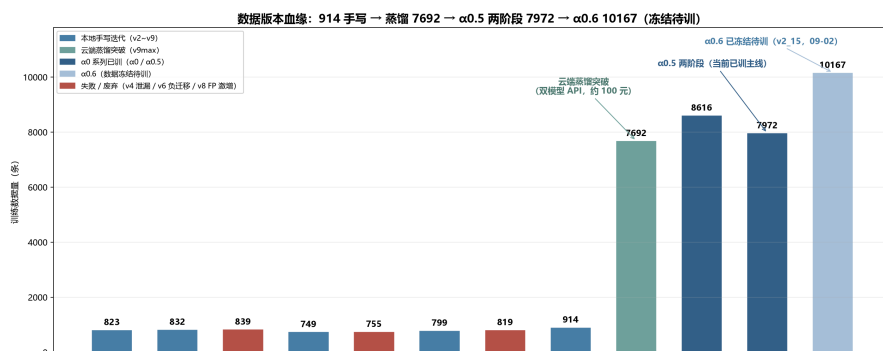


图 3-5 数据版本血缘（v2 → v3 → v4(废弃) → v5 → v6(失败) → v7 → v8(失败) → v9 → v9max → $\alpha 0$ → $\alpha 0.5$ ；标注各版本的废弃/失败原因）

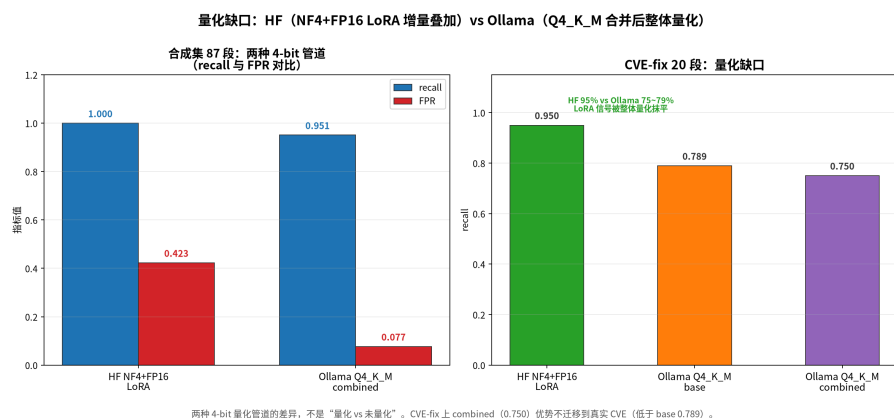


图 3-6 量化缺口对比（高精度形态与发布形态在真实 CVE 片段上的 recall 差异；已区分 base / combined 变体）

该图是“模型发布形态影响精度”的直接证据：两种 4-bit 管道（NF4 + FP16 LoRA 增量 vs 合并后整体 Q4_K_M）在真实 CVE 上存在约 16 个百分点的差距，而非“量化 vs 未量化”。

3.6.3 传统工具对比（14 段典型样本）

方法	recall	FPR	acc
LLM（本系统）	1.000	0.000	1.000
Bandit	0.833	0.500	0.750
Semgrep	0.750	0.000	0.786

关键案例：path_traversal_01.py 两个工具双漏报 (os.path.join 不做规范化需语义理解)，Bandit 把安全写法报 3 条误报。

3.6.4 训练成本与曲线

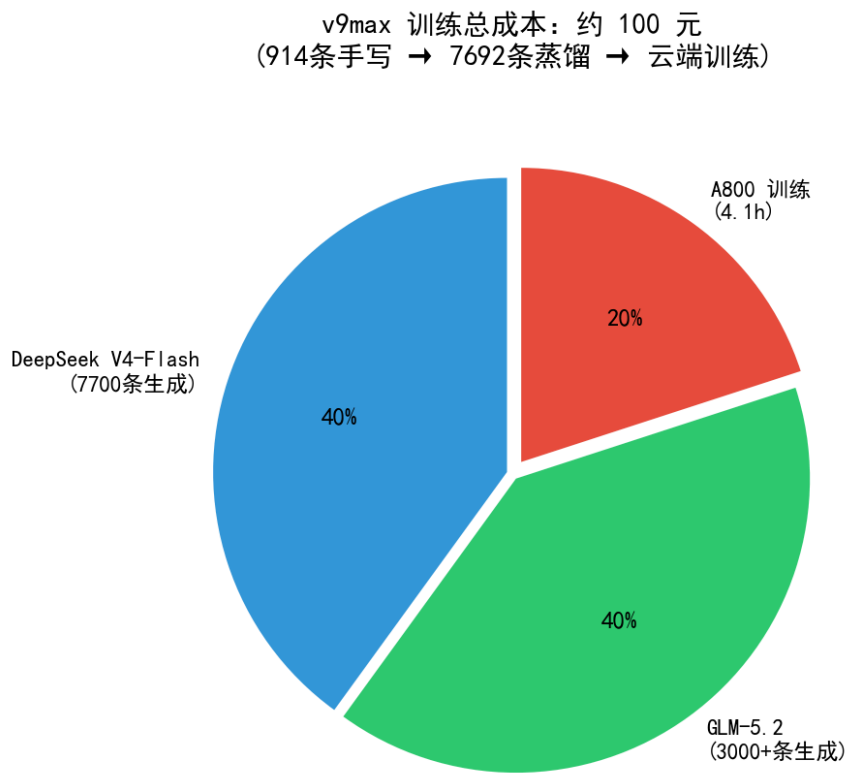


图 3-7 训练成本构成（数据蒸馏 + 云端训练合计约 100 元）

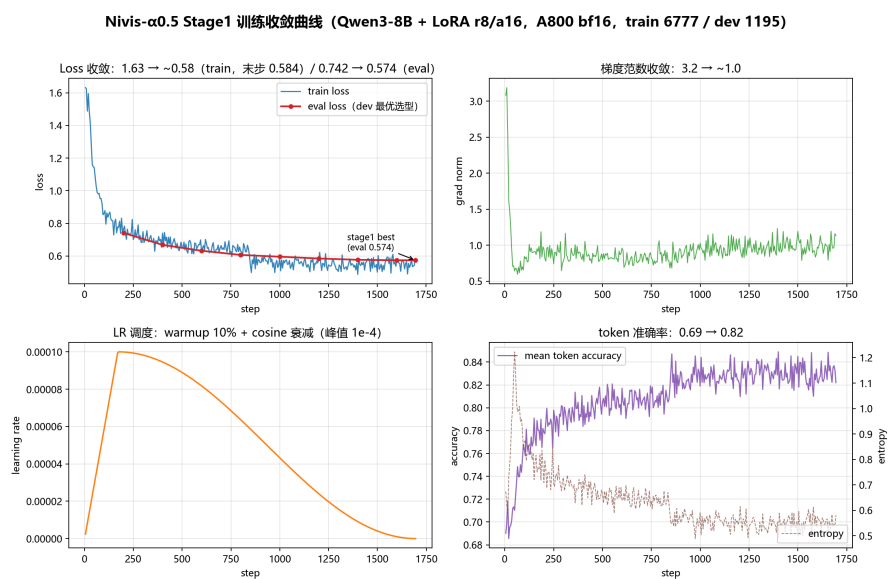


图 3-8 阶段训练曲线 (训练 / 验证 loss 随步数下降; train 6777 / dev 1195 条)

第四篇 详细设计说明书

本篇对应官方材料：详细设计说明书

4.1 Stage 1 召回设计

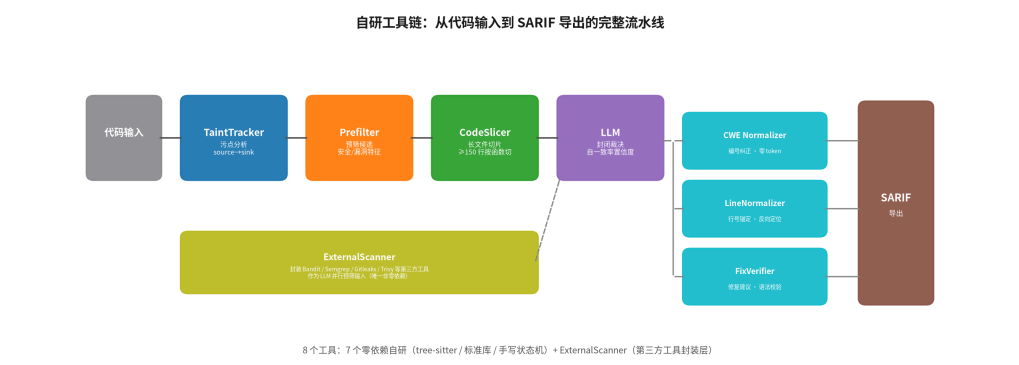


图 4-1 自研工具链流水线（从代码输入到 SARIF 导出的完整自研工具链；8 个工具中 7 个为零依赖实现，第 8 个 ExternalScanner 是唯一封装第三方工具的适配层）

4.1.1 自研工具 1：TaintTracker（污点分析）

基于 tree-sitter 的轻量污点分析（v2，线性数据流）：

- 按语句顺序扫描：source 赋值给变量 $v \rightarrow v$ 入污染集；赋值右值含污染变量 $\rightarrow$ 左值入集；
- 只有 sink 调用的参数里出现污染变量才报路径，不再做 source $\times$ sink 笛卡尔积；
- 消毒识别：仅类型无关函数 `int()/float()/bool()/shlex.quote()/intval()/filter_var()` 包裹、SQL 参数化 $\rightarrow$ 标记 sanitized；
- 单文件过程间摘要（两遍法）：第一遍生成摘要，第二遍在调用点拼接；
- 支持语言：Python / JavaScript / TypeScript / Java / PHP。

关键实现细节：

细节	设计要点
跨语言语义消歧	同一调用名在不同语言映射不同漏洞类型——JS/TS 中 <code>exec()</code> 对应 Command Injection，Python 中对应 Code Injection
危险度排序 + 截断	7 种漏洞类型按危险度赋值（Command/Code/反序列化=4，SQL/SSTI=3，XSS/路径=2），每作用域最多输出 50 条路径，按 <code>(-risk, sink_line, source_line)</code> 排序截断

细节	设计要点
语境安全判定	① Command Injection: subprocess 列表参数不经 shell 解析即安全; ② Template Injection: 纯 kwargs 值插值安全; ③ Path Traversal: 白名单 + abspath/realpath + startswith 防御链三者共存判安全
Web 框架参数自动 source	Python 检测 @app.route/@api.route/@router.get 装饰器; Java 检测 @RequestParam/@PathVariable/@RequestBody 等注解, 参数视为 source 种子
f-string 插值正确处理	逐字符扫描 + fstring_brace_depth 计数处理嵌套花括号, 整体保留 {interpolation} 不剥离, 避免污点传播断裂
跨语言赋值语法支持	Python walrus (:=) /named_expression/augmented_assignment、JS/TS/Java/PHP 的各类赋值节点逐一适配; 复合赋值运算符 (+= / -= / *= / /= / <<= 等 15 种) 单独处理——其右值为表达式, 若按普通赋值只取第一个子节点会漏掉污点来源
补充 source/sink 词表	PHP 的 ->loadHtml(归为 XSS sink 等 (按语言限定, 避免跨语言误匹配)

类型作用域陷阱 (历史教训): 全局消毒表曾把 `html.escape/urllib.quote/re.escape` 等仅对特定类型有效的函数应用于所有污染类型, 导致 SQL 拼接处包 `html.escape` 的流被误判“已消毒”而丢弃 (跨类型漏报); 且 `escape(` 会匹配 `unescape(` (反防御)。现只保留类型无关的消毒函数, HTML/URL 编码类消毒改由 LLM 裁决层判断。

4.1.2 自研工具 2: Prefilter (正则预筛)

- 命中安全模式且未命中漏洞特征 → 判安全; 仅命中漏洞特征 → 判漏洞; 两者都命中或都没命中 → 交 LLM 复核;
- 硬编码凭证标记: `has_secret_marker` 命中时不直接判漏洞, 而是抑制安全判定, 强制 LLM 复核;
- 两阶段架构中 **Prefilter** 只产出候选, 不短路终判。

关键实现细节:

细节	设计要点
规则元数据统一表	PREFILTER_RULE_INFO 每条规则统一关联 taint_type/cwe/risk/severity, scanner.py 与 two_stage_scanner.py 共用同一数据源, 避免规则映射漂移
字符串字面量感知	线性扫描定位配对括号时跟踪 in_str 状态跳过字符串内容

细节	设计要点
长文件安全规则门控	longfile_threshold=150, 超过此行数跳过安全规则只跑漏洞规则——防止“前半段参数化查询掩盖末尾隐藏漏洞”
高置信漏洞规则覆盖安全特征	pickle.loads/yaml.load 等高置信规则即使与安全特征共存也直接判漏洞
硬编码凭证正则修复	原 \b 词边界在 DB_PASSWORD 等下划线前缀上失效, 修复为 (?<![A-Za-z0-9]) 负向后行断言

4.1.3 自研工具 3: CodeSlicer (AST 代码切片)

文件 < 150 行不切片; ≥150 行按顶层函数/类方法切分, 每个切片含上下文头 + 类骨架 + 当前函数完整代码。

切片聚合语义: 任一 chunk 有漏洞 → 整文件判漏洞 (保守策略, 宁误报不漏报), 取风险最高的 chunk 结果。

批量解码正确性: generate_batch 必须左 padding (decoder-only 批量解码右 padding 时短序列的“最后一个 token”是 pad, 生成从 pad 位置继续产生不可靠输出); 批量失败 (如 OOM) 逐条回退而非整批报废。

4.1.4 自研工具 8: ExternalScanner (外部工具封装层)

封装 7 种第三方安全工具, 统一为五类扫描能力, 输出归一化为 ExternalFinding (rule_id/severity/type/位置):

能力	工具
SAST	Bandit / Semgrep
Taint	Semgrep taint
Secrets	Gitleaks / detect-secrets
SCA	pip-audit
IaC	Trivy config + trivy fs

关键实现细节:

- 本地规则优先 + 在线回退: 优先用本地化规则 (离线可用), 缺失回退官方 registry 包;
- 规则 ID 映射顺序敏感: codei → Code Injection 必须先于 cmdi 判断, 否则 codei_xxx 规则被误归为 cmdi;
- **Semgrep taint** 字段三级降级: 依次尝试 extra.metavars → extra.dataflow_trace → extra.taint_source/sink; semgrep OSS 1.172 taint JSON 不含以上任何字段, source 位置由 TaintTracker 或裁决层补全;
- **Trivy** 离线标志: trivy fs 必须带 --skip-db-update、trivy config 必须带 --skip-policy-update ——实测带参数 0.4s、不带 60.1s;

- 自研 **Semgrep** 召回规则六件套 (graduation_project/semgrep_rules/)：四条 `mode=taint` 型 (`sqli_taint` 28 pattern / `cmdi_taint` 32 / `xss_taint` 27 / `codei_taint` 18) + 两条结构化补位型 (`go_cmdi_shell` 22 / `c_cmdi_system` 8)，均整文件扫描（修复“长文件切片后 chunk A 的 source 与 chunk B 的 sink 被割裂”），注释明文“本规则只做召回，不做终判”。

结构化补位规则的动机（两例，均源于语言审计实测）：

规则	官方规则缺口	补位判据
<code>go_cmdi_shell</code>	官方 <code>dangerous-exec-cmd</code> 只匹配 <code>exec.Cmd{...}</code> 结构体字面量，完全不匹配 <code>exec.Command(...)</code> 函数调用——后者才是 Go 主流写法（语料池实测 <code>exec.Command</code> (4 个样本 vs <code>exec.Cmd</code> { 0 个）	检测「执行族 API + shell 解释器」组合结构
<code>c_cmdi_system</code>	安全审计包中 C 规则 9 条全指向内存安全（格式化串 / 随机数 / 竞争），命令注入零覆盖	检测 <code>system(cmd)</code> / <code>popen(cmd, mode)</code> / <code>execvp(cmd, ...)</code> 等执行族 API 且命令串非常量字面量；判据有 CERT C POS30-C (“Do not use the <code>system()</code> function”) 背书

`c_cmdi_system` 采用结构化检测而非 **taint**，原因：C 的典型链经 `snprintf(cmd, ..., argv[1])` 以输出参数写入，semgrep taint 无法跨函数参数传播（语言固有）。非常量命令串交给 Stage 2 裁决——LLM 能看到 `cmd` 的构建处与来源。

4.2 Stage 2 裁决设计

4.2.1 自一致率置信度

对同一 finding 以 `temperature > 0` 采样 $N=3$ 次，置信度 = 判真票数 / 有效票数。

无效票语义与文件级 **None** 传播：

- 分母用有效票 (`votes_true + votes_false`) 而非 N ——解析失败/推理出错的票不代表“否决”；
- 全部票无效时 `confidence=0`、`confirmed=False`，由聚合层判 **None**（需复核）；
- 文件级聚合统一把低置信确认/平票/低置信否决转成 `has_vulnerability=None`，防止“需人工复核”信号在汇总层被掩盖成“安全”。

4.2.2 裁决档位四分类

每个 finding 裁决后按 (`confirmed`, `confidence`) 定档：

档位	条件	去向
<code>confirmed_vulnerability</code>	判真且 <code>conf ≥ 0.8</code>	直接判漏洞
<code>confirmed_review</code>	判真但 <code>conf 0.5~0.8</code>	进人工复核清单

档位	条件	去向
dismissed_safe	判假且 $\text{conf} \geq 0.8$	排除
dismissed_review	判假但 $\text{conf} 0.5 \sim 0.8$	进人工复核清单

阈值只有一个： $\text{_CONF_AUTO} = 0.8$ 。 ≥ 0.8 直接采信，其余一律进人工复核（低置信确认、平票、解析失败同档处理）。

为什么不做“三档阈值”： $N=3$ 时自一致率的可能取值只有 $\{0, 1/3, 2/3, 1\}$ 四个点，任何介于 $1/3$ 与 $2/3$ 之间的边界都不产生行为差异。写一个不存在的 0.5 档只会让读者以为存在“低置信也算结论”的中间态，故代码中只保留一个边界。若将采样数 N 提高到 $5 \sim 10$ ，取值空间变密，才需要重新设计分档。

4.2.3 裁决上下文的行号精确对位

裁决 prompt 中 source/sink 的行号必须能直接对位源码。切片由“文件级上下文头 + 函数体”拼装：头部单独输出且不带行号，函数体从原文件按行截取并加 $N|$ 前缀；整文件 chunk 禁止拆分（否则代码以 $\backslash n$ 结尾时 split 多出的尾部空串会让首行被误判为 header）。

4.2.4 训练对齐裁决格式

裁决 prompt 使用训练对齐的 `has_vulnerability` 格式（与 $\alpha 0.5$ 训练格式一致）。历史缺陷（已修复）：解析器曾只认 `is_confirmed` 字段，而 prompt 要求输出 `has_vulnerability`——注释声称“双格式兼容”但代码从未实现该分支。后果：真实 CVE-fix 复杂输入下模型遵循 prompt 输出 $\rightarrow$ 3 票全 invalid $\rightarrow$ 候选裁决全转 review，recall 被误判为“工具零覆盖”。补齐 `has_vulnerability` 解析分支后，两阶段 CVE-fix recall 由 **0.556** 提升至 **0.882**（同组态前后对比：TP $5 \rightarrow 15$ ，review $11 \rightarrow 3$ ）。

数字口径提醒：真实 CVE-fix 片段（20 段）上的两阶段 recall 有两个数值并存，各自组态不同，不可混用：

数值	组态	出处
0.882	解析器修复后首轮（2026-08-20），本轮内部对比口径	上述“0.556 $\rightarrow$ 0.882”
0.941	官方口径 anchor 轮（2026-09-01），已纳入统一打分体系	见第三篇 §3.6.1

第三篇 §3.6.1 的系统级成绩表引用的是 **0.941**（**anchor** 口径）；本节描述的是解析器缺陷修复的效果对比。两者不矛盾，并排出现时须注明组态。

4.3 2.5 代信任层

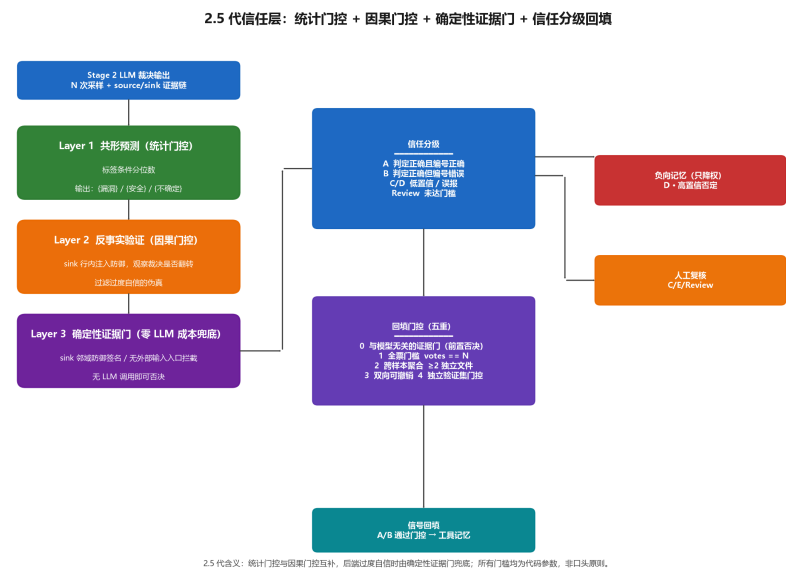


图 4-2 2.5 代信任层全流程 (Stage 2 裁决输出 → 三层门控 (共形预测 / 反事实实验证 / 确定性证据门) → 信任分级 A-E → 五重回场门控 → 信号回填 / 负向记忆 (只降权) / 人工复核)

信任层作用于 Stage 2 裁决之上，由三个门控层 + 一套分级回填机制构成：

层级	名称	类型
Layer 1	共形预测	统计门控
Layer 2	反事实实验证	因果门控
Layer 3	确定性证据门	静态门控 (零 LLM 成本)
回填	信任分级 A-E + 四重门控	写入侧防污染

图示的三层门控互补关系：统计门控与因果门控负责“判断质量”，确定性证据门负责在过度自信的推理后端上兜底；三者输出汇入 A-E 分级，只有 A/B 级能通过四重门控回填工具记忆。

4.3.1 确定性证据门（零 LLM 成本静态核验）

与推理后端无关、零 LLM 成本的确定性校验。设计动因：transformers (bf16) 后端 FP 全部是“全票但错”——共形/反事实实验证依赖的投票分歧信号在高精度后端上不出现 (Q4 量化噪声天然压制模式匹配型捷径，bf16 保留了过度自信)，统计门结构性失明。

门	判据
sink_defended	sink 邻域 (前 4 后 3 行) 已含该漏洞类型的已知防御特征 (参数化 execute / shlex.quote / html.escape / realpath / autoescape / json.loads) → 模型没识别已有防御 = 疑似模式匹配误报
no_input_entry	纯顶层字面量脚本且无外部输入入口 (request/input/argv/environ), 且仅对无任何函数/类定义的模块级脚本应用 → 污点无从产生

命中不否决 (不判 False), 仅把该 finding 排除出“直接判漏洞”依据转人工复核——门是保守的: 宁可 review 不错杀 TP (真漏洞的 sink 邻域不会出现完整防御特征, 真污点文件必有输入入口)。

生效语言范围 (如实声明): 本门目前仅覆盖 Python / JavaScript / TypeScript 三种语言——防御特征库只有这三种语言的签名。Java / PHP / Go 样本不走本门, 其兜底依赖 Layer 2 反事实验证。扩语言时须先补对应防御签名再扩白名单, 否则“签名查不到”会退化为无条件跳过, 等于白扩。

4.3.2 Layer 1: 共形预测 (统计门控)

机制: 以标签条件非一致性分数做三分类预测 {漏洞} / {安全} / {不确定}:

- 非一致性分数: $s(x, \text{漏洞}) = 1 - \text{判漏洞比例}$ 、 $s(x, \text{安全}) = \text{判漏洞比例}$;
- 全无效票特殊处理: $\text{valid} = 0 \rightarrow s = 1.0$ (全部票解析失败时对两个标签都“完全不像”, 天然落入不确定);
- 在带真实标签的校准集上拟合标签条件分位数阈值 ($\alpha=0.1$ 对应 90% 目标覆盖率), 阈值随模型导出持久化;
- 分位数线性插值 $\text{idx} = \text{level} * (\text{len} + 1) - 1$, predict 用 $s \leq q + 1e-9$ 浮点容差;
- 加载失败可审计降级: 文件缺失或解析失败时返回未校准状态, 预测全部落入“不确定”并回退旧投票逻辑, 而不是抛异常中断扫描。

校准的 **exchangeability** 前置条件与当前状态 (如实声明):

覆盖率保证成立的前提是“校准集与生产分布可交换”。项目在自查中发现: 此前生产端自动加载的校准文件是在同一测试集上拟合的 (in-sample), 该前提被破坏、“90% 覆盖率”不成立。处置是把该文件移出自动加载路径 (保留存档、不再参与生产推理), 并让扫描器在未显式指定校准源时保持未校准。

因此当前生产组态下 Layer 1 处于未校准状态: 不做统计裁决, 全部判定交由后续门控与投票逻辑处理。这一取舍的理由是: 宁可宣称一个不成立的统计保证, 也不用一个来源不干净的阈值去影响判定。

定位说明: 本层的价值在于它提供了一个与判定阈值解耦的统计门控接口——一旦获得独立同分布的校准集 (例如用滚动开发集校准), 即可在不改动上下游的前提下启用。它不构成本系统结论正确性的必要条件。

4.3.3 Layer 2: 反事实验证（因果门控）

在 sink 行内注入防御后重判，区分“真理解”与“模式匹配”：

- 防御模板最小化设计：仅对 sink 行内做消毒包裹（f-string 加 shlex.quote、字符串拼接包裹、execute 拼接改参数化、单引号拼接 replace("'", "'')、open 包 realpath、补 autoescape、pickle.loads 改 json.loads），不破坏代码结构保证语法正确；
- **SQL 扰动“语义保持”**：改参数化时保留原查询字符串（.execute(原查询串, (变量,)))——原实现把查询整体改写会让模型误判“换了查询逻辑”而非“加了参数化”，扰动失去因果检验意义；
- **already_defended** 范围限定：判定“原始代码是否已含同类防御”限定在 source→sink 区间（source 未知时退化为 sink 前 15 行窗口），不做全文搜索；
- **finding 级裁决**：验证用 finding 级 triage prompt（只裁决本 finding），不用开放扫描 prompt；
- 翻转判定用单次低温裁决（不 N 采样，省 5 倍推理）；
- 仅 **already_defended** 才降级：真漏洞扰动后仍判漏洞（flipped=False）是正确行为，绝不能降级。

4.3.4 信任分级回填（工具-模型自适应闭环）

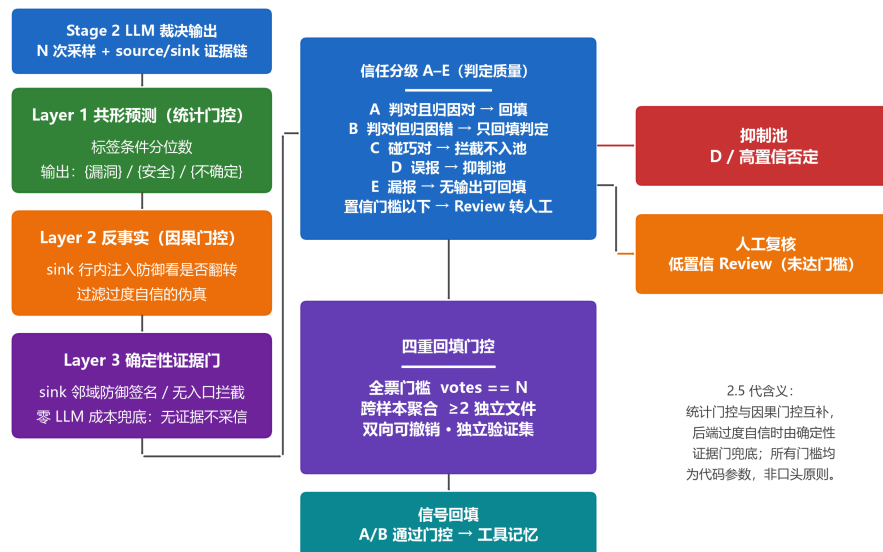


图 4-3 信任分级回填门控（A-E 分级 → 五重回填门控 → 回填工具记忆 / 转为负向记忆（只降权） / 转人工复核 三条去向）

模型输出按判定质量分 A-E 级，只有 A/B 级（判定正确）可回填，C/D 级反向使用（D 级误报转为负向记忆）。回填写入发生在“最终结论确定之后”——即聚合完成、三道门控都已生效之后，用最终采信结论而非模型原始投票作为写入依据。

回填门控（五重，前四重防“模型不自信”，第五重防“模型自信但错”）：

0. 与模型无关的证据门（前置否决）：最终结论为“需人工复核”、或该判定被证据门拦截、或反事实验证判存疑时，一律拒收——这类判定正是“模型可能错了”的信号，不能作为工具记忆的依据；
1. 全票门槛（votes == N）；
2. 跨样本聚合（ ≥ 2 独立文件）；
3. 双向可撤销（高置信否定清零已积累确认）；
4. 独立验证集门控（待学习池特征须经独立验证集复验才转正）。

门控 0 的必要性来自一个实测现象：在高精度推理后端上，模型的误报形态恰恰是“三次采样全票确认”——全票反映的是“模型很自信”，与“模型很可能对”是两件事。只用投票一致性做门槛，等于用一个与已知失效模式正交的量去设防，因此必须叠加与模型无关的证据。

关键实现细节：

细节	设计要点
双向可撤销	高置信确认 ≥ 2 个独立文件且确认数 > 否定数 → 撤销此前积累的负向记忆（修复前确认分支无撤销路径，模型两次全票假阴性就能永久杀死一条规则召回）
单次否定不降权	否定必须 ≥ 2 个独立文件才形成有效负向记忆；单次否定只累加计数
负向记忆只降权、不剔除	模型判定“这条规则在这个文件上是误报”时，只降低该规则在本文件的候选排序权重（priority_factor），不从候选集合中删除。原因：规则是否误报取决于文件而非规则本身——同一条规则在 A 文件上是不容置疑的证据、在 B 文件上可能是正确告警；按规则级“见到就跳过”会制造样本级静默盲区。该设计曾因规则级过滤导致工具召回系统性崩塌（历史事故），故把动作从“剔除”降级为“降权”，并保留完整留痕
类型校正去重计数	corrected_type_counts 按 (file, corrected_type) 去重，读写端口径一致
待学习池	收录“工具漏召但 LLM 全票判中”的代码特征，须经独立验证集复验才转正
正向 boost	已回填高置信信号的规则 boost_priority 返回 1.0 + confidence，Stage 1 候选按此排序——模型回填不只过滤，还正向影响候选顺序
确定性工具免疫	模型意见仅作用于裁决档（taint/prefilter/sast/iac）；直出档（secret/sca，确定性工具）不参与回填、也不受模型意见影响
自动持久化	record() 变更后自动 save()（原子写：tmp + os.replace）

闭环小结：门槛参数写入代码、读写端语义一致、未达门槛保留原值（防止模型错误判定污染确定性工具）；三个去向的决策依据均为最终采信结论——确认 → 排序加权，否定 → 按文件降权，类型校正 → 严格条件下更新映射。没有任何一条路径会降低工具层的召回面。

4.3.5 防锚定的三层结构

同一份错误提示，Q4 后端 0/3 全票否决、bf16 后端 3/0 全票确认——“全票但错”说明模型对工具锚点过度顺从。应对不是全局降服从性，而是条件性服从：

层	机制
推理层	prompt 中低信任位置型规则显式注入“⚠ 此告警来自位置型规则（无 source→sink 数据流证据链），历史误报率高.....不要顺着工具标注的方向走”；prefilter 标“可信度中等”；带 source→sink 链的标注“可信度较高”
锚点来源分级	高信任标注只在 source/sink 确实由工具产出时授予。位置型候选本身没有 source/sink 文本，模型判真时会自述锚点并回写到候选上——若不加分，模型自述的行号会被下游当成“工具引擎产出的证据链”授予高信任，形成自我确认。实现上为候选记录锚点来源（tool / model_vote）并分级注入措辞，同时为反事实验证保留一份锚点回填前的候选快照
聚合层	确定性证据门
回填层	信号注册表

4.3.6 多数票类型投票（回声票加权）

文件级聚合多个裁决的 vulnerability_type 时，投票键 = (总票数，独立票数，该类型所在 finding 的最高严重度)。

独立票 = 模型输出类型 ≠ 工具 taint_type 归一化结果——“回声票”只是复读工具标注，无独立信息量（工具标 XSS、模型跟着输出 CWE-79 是回声；工具标 B324、模型独立判出 CWE-327 是独立判断，平票时独立判断胜出）。

文件级存在性判定：任一 finding 多数票判真即文件判真（不因无关 finding 否决而对冲），但排除两类：① category=="llm" 的合成 finding；② evidence_gate 拦截的判真。

4.4 复核兜底机制

工具规则覆盖不可能 100%，工具盲区在裁决路径上会静默放行（裁决式任务只问“工具告警是否为真”，模型不会自主发现工具没召回的漏洞）。为此系统设有两个复核出口：

出口	触发条件	判定
① 无候选复核	文件无任何工具候选时触发；具体策略由 no_candidate_mode 控制，分三档组态（见 §4.4.1）	复核命中且全票判真 → 采信为漏洞

出口	触发条件	判定
② 裁决全否决兜底复核	全部候选被裁决否决、文件将被判安全时，再做一次全文件兜底复核	命中且全票判真 → 采信为漏洞

术语说明：出口① 内部有 `full_recheck` / `targeted` / `sampled` 三档组态（那是“同一出口的三种策略”，不是三个出口）；系统对外共两个复核出口。

4. 4. 1 无候选复核的三档组态

组态	生效场景	行为
<code>targeted</code> （生产默认）	所有生产入口：单文件分析、批量上传、GitHub 仓库、URL 抓取	三档定向复核：按“文件风险分 × 盲区命中”分配注意力，零盲区文件不送 LLM——替代全量复核（后者成本随文件规模线性增长，且大多花在零风险文件上）
<code>full_recheck</code>	vLLM 端点；以及显式指定用于演示 / 消融	每个无候选文件全量 LLM 复核，消除“无证据判安全”静默放行
<code>sampled</code>	扫描器类默认与评估组态	10% 抽样复核（监控工具层漏报率，省算力）

场景 → 组态对照（代码核实）：

扫描场景	默认组态	可覆盖方式
单文件分析（粘贴 / 上传单文件）	<code>targeted</code>	<code>VULN_SCANNER_BATCH_RECHECK_MODE</code>
批量上传	<code>targeted</code>	同上
GitHub 仓库	<code>targeted</code>	同上
URL 抓取扫描	<code>targeted</code> （跟随批量）	<code>VULN_SCANNER_URL_RECHECK_MODE</code>
vLLM 端点	<code>full_recheck</code>	代码内固定
评估 / 消融	<code>sampled</code> （类默认）	评估脚本参数

三个生产入口共用同一组态常量——`_two_stage_scan()` 的参数默认值即 `BATCH_NO_CANDIDATE_MODE`（默认 `targeted`），不做“按入口分场景”的差异处理。这样设计是为了消除“行为随调用顺序漂移”：早期实现让请求级参数覆盖共享配置后不回滚，导致跑过一次仓库扫描后，后续单文件扫描会静默沿用仓库扫描的组态。

口径提醒：三档必须严格区分——生产默认是 `targeted`；若引用“10% 抽样复核”，须注明其为类默认 / 评估组态；`full_recheck` 只在 `vLLM` 端点与显式指定时出现。

4.4.2 无候选复核的判定门槛

抽样算法细节（反直觉设计）：

- 复核票数上限 3：`n = max(1, min(3, self.n_samples))`——复核最多 3 票，是“最便宜的兜底”，用更少票换取速度；
- 温度取 **max**：`temperature = max(self.temperature, 0.7) if n > 1`——多票采样需足够温度打破同模态重复；
- 计数器口径防污染：裁决全否决兜底调用时 `count_monitor=False`，不计入 `no_candidate_totalo`。

4.4.3 `targeted` 组态的三档定向复核

`targeted` 以「文件风险分 × 盲区命中」把无候选文件分为三档，把复核预算集中到真正有攻击面的文件上：

档	判定条件	复核动作
A 档	有盲区 且（风险分 ≥ 阈值，或 高优先级盲区命中 ≥ 2 条）	盲区片段 × <code>min(3, n)</code> 票（与全量复核同置信级别）
B 档	有盲区但低危；或 零盲区但高风险分	片段 × 1 票。单票判真不具采信资格，一律转人工复核——“一票通过”是没人反对，不是多票一致；而无工具证据的采信必须是最高置信级别
C 档	零盲区 且 低风险分	不送 LLM，显式留痕“未获得复核预算”；但仍按 10% 抽样，抽样命中则降级为 B 档

为什么“零盲区”不能直接等于“不送 LLM”：盲区规则覆盖的是工具写不了规则的形态（越权、过滤可否绕过），而 SQL 注入 / 命令注入 / 路径穿越 / XSS 这类工具本该召回的形态并不在盲区表里。若一律按“零盲区 → 跳过”处理，一旦工具因规则不覆盖、语言不支持或新框架而零召回，这些漏洞就被静默放行——正是本系统一贯要消除的静默性。因此加两道保险：① 风险分高（有外部源 + sink + 入口点 = 有实际攻击面）却零召回，是工具失效的信号，比“文件没内容”更值得复核，故降 B 档；② C 档按 10% 抽样复核，在线估计零盲区文件的漏报率。

风险分阈值：含外部源 + sink + 入口点的业务文件典型得分 20~40，纯工具层 / 数据传输对象为负分，高优先级档阈值为 12 分。

省时的两个来源（可分别关闭以做消融）：① C 档省掉整次调用（大仓库的主要收益）；② A/B 档把“整文件”换成“盲区片段”（自检实测 5575 → 285 字符，约 5%），模型从“读 1000 行找漏洞”变成“看几个带行号的短片段判漏洞”。

在线漏报率估计（分层口径）：模块级计数器把复核结果按抽样机制分层统计。只有 C 档按比例随机抽样命中而升级的那部分是无偏随机样本，可用来估计“工具层漏报率”（并给出二项比例的置信区间，样本不足时只报区间不报点估计）；A/B 档是按“风险分 × 盲区”定向挑选的样本，命中率只能称为“定向复核命中率”，不能当作漏报率——否则会把“专挑高风险文件复核”的选择偏差算成工具的能力缺陷。分层后，这个指标从“看似精确的单一数字”变成“随机层给率 + 定向层给命中率”的两组可引用数字，把工具召回漂移变成可观测且可辩护的指标。

复核采信形态校验（防止 LLM 在工具盲区上瞎判）：

- 注入型漏洞（有标准 sink 和标准防御的）必须满足 ① 代码存在该类型标准 sink 特征；② sink 邻域（±8 行）无该类型标准防御 → 任一不满足转人工复核；
- 缺失型/其他漏洞（CSRF/认证缺失/弱密码/硬编码/竞态/XXE 等无确定性验证手段的）不做形态校验，全票采信并如实标注“模型语义兜底”；
- 跨文件调用（sink 在外部文件时“调用导入函数”= 可能存在外部 sink）不误拦。

4.5 关键工程机制

4.5.1 优先级调度器

基于 heapq 优先级队列，交互式扫描（HIGH）优先于批量扫描（LOW）：

- ScanTask dataclass 按 (priority, seq) 排序，优先级相同时 FIFO；
- 单工作线程与各后端串行并发模型对齐，Python 层显式排队；
- 每 client_id 最多排 8 个任务，防止单一批量扫描霸占队列；
- 工作线程通过 loop.call_soon_threadsafe 把结果回填到 asyncio.Future；
- 请求头 X-Scan-Scope 让调度器把插件交互式扫描排 HIGH、批量排 LOW；
- 配额桶由服务端事实决定：批量入口（单文件批量 / 仓库 / URL）各自生成一个服务端批次标识作为配额分组键，客户端无法通过轮换请求头把自己的任务拆成多个桶绕过每客户端上限；X-Client-Type 亦做白名单收口，未知取值统一归入默认桶。

关停时的任务收尾：调度器停止工作线程前，会把仍在队列中的任务整体置为异常，并对关停后到达的新请求立即返回异常。原因：工作线程退出后队列中任务永远不会再被取走执行，若 Future 不回填，端点的 await 会永久挂起，拖住服务优雅停机。这属于“不留悬挂状态”的一贯原则在关停路径上的体现。

4.5.2 约束解码兜底

CoT + JSON 解析失败时，用 format=json 约束解码重试，数学上保证输出可解析。这是 parse_fail 从 18/87 降到 0/87 的关键机制之一。

注：该机制是 Ollama 后端独有能力；LlamaCPP 后端未启用 GBNF 语法约束，退化为普通 generate + 警告。

4.5.3 手写括号深度匹配 JSON 解析

LLM 输出解析的“最后一道防线”。非贪婪正则则在遇到 "reason": "使用了 {username} 导致 SSTI" 时会提前截断，而手写状态机跟踪字符串内/外的花括号深度，收集所有平衡闭合的 {...} 候选逐个 json.loads。

4.5.4 CWE 编号确定性纠正

模型对漏洞“语义分类对、编号记错”（如 SQL 注入写成 CWE-29）是纯记忆错误，纠正模块在输出后做确定性查表纠正，零 token 开销。幂等且对表外长尾原样放行。

细节	设计要点
关键词排序防子串歧义	按“更具体优先”排列（NoSQL 在 SQL 之前、CSRF 在 XSS 之前），避免子串误匹配
短关键词词边界保护	cmdi/codei/logi 等用词边界正则，避免匹配 cmdispatch/logical
evidence 守卫四级优先级	① 字段命中表内语义关键词 → 直接用；② 字段已含明确 CWE 编号 → 尊重模型判断；③ 无编号 → 用 evidence 二次提取；④ 都未命中 → 原样返回
父子族匹配双向遍历	同时遍历 reported→expected 与 expected→reported 父链

4.5.5 行号内容锚定纠正

模型输出的 source / sink / fix_suggestion 采用“行号锚定”格式（如 line 7: request.args.get('file')）。行号是纯“数行”任务，模型容易数错，但 line N: 后面的行文本内容通常可靠。本机制用内容在源文件中定位真实行号并覆盖，零 token 开销：

- 支持 line N: / L N: / 第 N 行: 三种锚点形态，纠正后统一输出 line N:；
- 三级评分：整串相等/代码骨架包含 → 1.0；最长无空白代码片段包含 → 0.95；去中文后相似度兜底；
- 高分优先、同分取距原始行号最近者（行号幻觉通常是“差几行”而非跨文件乱跳）；
- 匹配不到可靠锚点（< 0.6）原样保留，不做破坏性覆盖。

4.5.6 修复建议验证

验证模型输出的修复建议是否语法正确、是否真正移除危险模式。修复建议为“行号锚定的单行局部建议”，验证器主职是抓幻觉行号。

细节	设计要点
补充危险模式表	15 条补充模式，覆盖 Java 命令执行 / Java SQL 拼接 / Java 路径拼接 / Flask SSTI 拼接 / 弱哈希 (Java 与 Python 两侧) / SSRF 拼接 / 开放重定向拼接 / SSTI 组合特征；选取原则是“必须是危险用法（拼接、弱算法）而非 API 存在”——否则修复后的安全写法仍会命中，导致“修复无效”误判

细节	设计要点
Java 双候选包壳策略	生成两个候选编译单元（声明放类体 / 语句放 main 方法体），两种都失败才判语法不合法
JS CJS→ESM 回退	node --check 默认按 CJS 解析，import/export 会误报，CJS 失败后自动用 --input-type=module 重试
extract_code 取最后一个代码块	fix_suggestion 含多个代码围栏时取最后一个

4.5.7 SARIF 2.1.0 标准化导出

与 GitHub Code Scanning、VSCode Problems、JetBrains Qodana 原生互通。详见第三篇 §3.4.3。

4.5.8 多模型投票聚合规则

- **has_vulnerability**：多数票决定；50/50 平票 → None（转人工复核）；
- **consensus**：全票一致 = unanimous / 有少数派 = majority / 平票 = split；
- **agreement_ratio**：多数方票数 / 有效票数（排除 error 的 None 票）；
- 结构化字段：取自多数方首个模型；平票时结构化字段仍取判真侧（供复核参考），但结论本身是“需人工复核”；
- **explanation/raw_output**：取自首个与最终结论一致的模型；
- 全部模型失败 → has_vulnerability=None、error="all models failed"。

平票为什么不判漏洞：两个模型各执一词时，“证据最强对立”恰恰是最不该给强结论的情形。系统在 Stage 2 的平票口径是“转人工复核”，多模型通道必须与之一致——否则会出现“同一系统两套平票语义”，且是最弱证据给出最强结论。

通道边界：多模型投票仅对 Ollama 后端开放。进程内后端每次只能加载一个本地模型，传入多个模型名会指向同一模型（“三票同源”假投票），因此后端以 501 门控、前端按健康检查中的模型管理能力禁用该选项。

顺序加载/卸载策略：逐个模型加载 → 扫描 → 立即卸载（keep_alive=0）→ 加载下一个，任一时刻显存中只驻留一个模型。finally 块保证即使中途异常也能卸载。

4.5.9 模型管理能力门控（501 而非 500）

进程内后端没有运行时拉取/删除能力，/api/models/* 返回 501 并附能力清单。501 的语义是“我知道这个功能是什么，但当前后端不支持”，让前端可以优雅降级（如隐藏“拉取模型”按钮）。

4.5.10 多后端抽象与精度权衡

四种后端工厂按类型构建对应客户端。后端为前端提供检测报告式信息（基座量化位宽、LoRA 是否被量化、运行设备等），让用户直观理解“为何 HF/LoRA FP16 与 Ollama Q4 发布态是两档精度”。

merge 通道永久关闭：transformers 后端 `self.merge = False` 硬编码，环境变量与构造参数均被忽略。原因（代码注释明文）：NF4 4bit 基座上 `merge_and_unload` 把 LoRA 增量折回 4bit 存储，归因能力显著退化（strict 0.709 → 0.525）。

4.5.11 训练/推理一致性保证

- 模型注册表每个模型绑定 `prompt_variant`，推理时自动选择对应的 `system prompt`；
- `:latest` 标签归一化避免误判“未安装”重复拉取；
- `deprecated` 字段标记过时模型，UI 标记“已过时”但仍可使用；
- 回退链路：未登记模型回退基础 `prompt`；
- 裁决/验证 **prompt** 一致性与运行时同步：训练对齐裁决要透传到每一处 `prompt` 组装（曾出现 `system` 已是训练格式、裁决 `user` 却问旧字段的不一致）；切模型后 `system_prompt` 需运行时统一刷新（`sync_runtime()`）——跨组件共享的 `prompt` 状态应在运行时统一刷新，不能依赖构造期捕获。

4.5.12 URL 抓取的多层防护

层	措施
协议白名单	仅允许 http/https
IP 分类	拦截内网/回环/链路本地/保留地址
DNS rebinding 缓解	双重解析一致性校验
重定向控制	每一跳重新校验，最多 5 跳
响应体限制	流式读取，超过 2 MB 截断
代理感知的 SSRF 校验	检测到将经 HTTP 代理发出时，跳过 IP 级校验（见下）

代理感知的校验豁免：代理模式下 `requests` 把主机名原样交给代理、由代理解析并建连——本机 `getaddrinfo` 的结果与实际连接目标无关，IP 级校验（内网地址拦截、rebinding 复检）只在直连时有意义。若在代理下照常校验，会出现“本机 DNS 被污染 → 解析为空 → 误拦可达站点”的假阴性（大陆网络下部分 CDN 域名即属此情况，经代理实测 HTTP 200）。因此实现上先判断该请求是否经代理，再决定是否施加 IP 级校验。

4.5.12.1 库文件与压缩产物的过滤策略

URL 扫描面向的是站点自有代码，第三方库与打包产物不属于攻击面，进入 LLM 裁决只会消耗预算并产生假阳性。因此在抓取入口设三道过滤：

过滤	判据	实现要点
第三方库 URL	主机名 / 路径 / 文件名中的库名关键词	hostname 匹配与 path 匹配必须分正则：hostname 永不含 /，带路径的模式（如 google.com/recaptcha、gitee.com/libs）写进 hostname 正则永不命中——此处曾因未区分而静默失效
自托管库副本	库名 + 可选“扩展名尾巴”	只写 jquery\min.js 会漏掉 /resources/js/bootstrap.min.js 这类站点把 CDN 库下载到自己服务器的副本（形态与自有代码相同）；按“库本身非攻击面”原则同样过滤
压缩/打包产物	内容启发式（无网络请求）：带 sourceMappingURL 注释；或平均行长 > 250 字符；或过半行单行 > 2000 字符（内容 ≥ 4KB 时判定）	压缩产物标识符被破坏、变量名全毁，LLM 语义裁决价值极低，召回多为打包器形态假阳性（实测某仓库 53/57 个 webpack chunk 全部进裁决、逐条被驳回，预算几乎白烧）。默认跳过并显式回报计数（可设环境变量恢复全量扫描）——遵循系统“不静默丢弃”的一贯原则

被跳过的文件数会显式回报给用户，而非静默丢弃；这与 §4.4「复核出口显式留痕」、第四篇 §4.5.17「预算外文件必须可审计」属同一设计原则。

4.5.13 GitHub 仓库扫描的沙箱约束

git clone --depth 1 浅克隆到 tempfile.mkdtemp，120 秒超时，GIT_TERMINAL_PROMPT=0 禁交互凭据提示，扫描后 shutil.rmtree 清理；跳过目录用路径段精确匹配而非子串匹配（子串匹配会误伤 x.gitlab 等合法目录名）。

遍历与预算分离：遍历阶段只做“防内存爆炸”的硬上限（文件数 3000 / 总字节 200 MB），不做按遍历顺序的截断——扫描谁由风险分决定（见 §4.5.18）。两者必须有足够差距：若收集上限不够大，风险排序只能在一个被顺序截断的小集合里排，靠后的认证 / 接口目录可能压根没被收集到。为避免这条不变量随时间失效，代码把它做成导入期断言：请求层文件数上界是一个独立常量，与收集上限绑定校验，一旦有人把上界调到接近收集上限，服务启动即失败而不是静默退化。

4.5.14 向量知识库设计

存储选型与检索参数（BAAI/bge-m3 多语言 embedding、HF_HUB_OFFLINE 强制离线、三级模型路径解析、集合与检索参数）见第五篇 §5.3。本节只关心工程行为：Chroma 向量库延迟到首次使用才初始化，初始化失败静默降级为纯 LLM 判定并记忆失败，避免反复重试。

4.5.15 输入限额的前后端双端一致防线

后 端 定 义 MAX_CODE_CHARS=2M / MAX_BATCH_FILES=200 / MAX_SINGLE_FILE_BYTES=2MB /
MAX_BATCH_TOTAL_BYTES=10MB，超限返回 413（非 400/500）；前端用同值先做客户端拦截（超限 toast“已跳过 N 个文件”）。本地单机服务也做输入限额 + 前后端双重校验，且用 413 精确表达“请求实体超限”语义。

4.5.16 国内镜像下载的工程闭环

细节	设计要点
HF_ENDPOINT import 时序	在任何 <code>huggingface_hub import</code> 之前设置——hf 的 endpoint 在首次 import 时固定，这是唯一可靠时机
NO_PROXY 强制直连	把镜像域名追加进 NO_PROXY，避免代理秒断 + 白耗 GB 级代理流量
socks 规范化	把裸 <code>socks://</code> 修正为 <code>socks5://</code> ，否则抛 <code>Unknown scheme for proxy URL</code>
断点续传 + 退避重试	HTTP Range 续传写入 <code>.incomplete</code> 后缀；416（Range 越界）时目标文件已存在直接判完成；网络抖动指数退避最多 5 次； <code>downloaded == total</code> 不符则删除重来
下载源白名单	模型下载端点只接受可信模型来源域名，并对目标地址再做一次“解析后 IP 分类”校验（防域名被劫持指向内网）。此前该端点接受任意 URL，可被用来探测内网服务
下载大小上限	单文件 8 GB 上限：Content-Length 已知且超限时直接拒绝（不留半成品），流式累计超限时中止并删除半成品；超限属确定性错误，不进入退避重试
代理环境变量的并发原子性	镜像直连配置（NO_PROXY）与代理协议规范化都是对进程级环境变量的“读—合并—写”。两个下载并发时无锁的读改写会互相覆盖丢项，故用进程级锁串行化该临界区（临界区仅内存操作）

4.5.16.1 进程内推理的墙钟超时（可用性底线）

问题：四种推理后端共用同一个 `generate(..., timeout=...)` 调用契约，但进程内（Transformers）后端的生成发生在当前进程里，请求层超时对它是无效参数——一旦生成卡住（显存抖动、驱动挂起、超长输入触发换页），它会一直占住推理线程。而扫描调度器只有一个工作线程，且明确声明“线程无法安全强杀”，结果是整个服务停止响应，用户只看到进度条一直转。

处置：为进程内生成加墙钟停止条件——在解码循环中检查累计耗时，超时即停止生成并返回一个显式错误结果；错误经既有的“无效票”路径进入复核，不需要改动上层任何逻辑。同时把裁决路径的生成长度上限收紧到与裁决输出规模匹配的值（裁决只输出固定字段 JSON），既降低卡死时的浪费，也降低生成途中开始复读的概率。

并把不可见变可见：调度器的“疑似卡死”标记（执行时长超过阈值）此前只存在于队列状态接口里，仪表盘与启动器都读不到。现提升为健康检查的顶层降级标记，使“引擎可能卡住”成为可被界面与运维感知的状态。

4.5.17 工具层盲区定位（行级提醒，不产生 finding）

传统工具只能召回可规则化的漏洞形态；有一大类风险“规则写不了”——授权/越权（IDOR，需业务语义）、过滤是否可绕过（取决于下游）、跨函数跨文件污点、加密用法是否安全、隐式信任边界。这些若写成规则，要么漏得厉害，要么误报爆炸。

设计：把它们做成行级盲区提醒注入 **prompt**——只描述“工具看不到什么”，把判断权完整交给模型。三条硬约束（缺一不可，否则就是换皮的告警）：

1. 不产生 **finding**：盲区不进候选集、不进裁决、不影响漏洞判定结果；
2. 永不进信号注册表：盲区被模型否定是正常且高频的（本就是低置信提示），一旦允许其进入工具负向记忆，该机制会在几轮扫描后自我清空；
3. 中性措辞：统一用“工具无法判定 X，请确认 X”，绝不写“此处存在 XX 漏洞”——后者会锚定模型、抬高判真率。

知识来源为先验分类（公开漏洞形态目录），与“从样本挖掘 + 独立验证集审批”的学习通道严格分离，防止样本过拟合。纯确定性：同输入必同输出。默认开启，可通过环境变量关闭以供消融对比。

4.5.18 文件级风险打分与注意力预算调度

修正的架构缺口：仓库/URL 批量扫描此前按目录遍历顺序取前 N 个文件即停止——这是盲目截断。大仓库中工具目录、数据模型目录常排在认证、接口目录之前，有限名额可能被低危文件占满，高危文件压根没进扫描（此时优化“单文件复核多快”没有意义）。

方案为纯确定性的文件风险打分 + 预算分配：

- 打分只用语言习语级知识（路径语义：认证/接口/支付/加密等 + 公开漏洞形态词表），不从任何测试样本挖掘——样本挖掘属于独立的学习通道（须经独立验证集审批），两者严格分离以防过拟合；
- 分配在给定预算（文件数 / 总字符数）下按分从高到低取；未覆盖的文件显式回报（`BudgetPlan.uncovered`），绝不静默丢弃——与本系统“消除静默性”的一贯原则（被抑制、被剔除的候选均留痕）同构：预算外文件也是“没扫到”的一种，必须可被审计；
- 可选同构折叠：大仓库中常有几十个近重复的模型/数据传输对象文件，按结构指纹折叠后只扫代表文件，其余记入 `duplicates_folded`。

纯确定性（同输入必同输出、无随机、无 LLM）→ 可复现、可审计、可作为消融实验对象。

4.5.19 列表形式命令调用的安全性判定（唯一入口）

修正的历史缺陷：多处实现曾把“列表形式 subprocess 调用”无条件当作安全/防御，导致 subprocess.run(["sh","-c",user])、find -exec、git --upload-pack 等由解释器执行载荷的真注入在召回层、证据门/复核门、安全白名单被系统性拦截（漏报）。

规则：列表形式默认不经 shell 解释（构成防御），但参数中出现 **shell** / 解释器或执行语义参数时，载荷会被该解释器执行，仍构成命令注入。该判定封装为唯一入口供各层（反事实验证、污点分析、schema、prompts）复用，禁止在各层自行复制“列表形式 = 安全”的局部判断——单一事实来源，避免判定漂移。

4.6 设计依据

4.6.1 路径依赖与部署设计

设计	依据
一键启动脚本依赖装进当前解释器同环境运行	避免依赖分裂（torch 对了却缺 tree_sitter / 工具装到别的环境）
Windows bat 不切 UTF-8 代码页	脚本路径可能含中文，切 UTF-8 后 cmd 仍按 ANSI 解析 %~dp0 会“吃掉”命令行尾部；保持默认代码页 + echo 文本用 ASCII
torch 构建硬件分档	按 compute capability 分档（12→RTX50、8.9→40 系、7.5→20 系、6.x→10 系）；AMD 按 RDNA 代数选 rocm 版本；<6GB 显存走 CPU torch
同版本 CPU→GPU 替换先卸载再装	否则 pip 认为已满足
不主进程 import torch	避免清理前就弹 DLL 入口点错误窗口

4.6.2 llama-cpp-python 的两个真实坑

- 坑 1：llama-cpp-python 0.3.34 加载 **safetensors LoRA** 推理会卡死（**hang**），GGUF LoRA 才是原生格式 → 优先选 adapter_model.gguf；
- 坑 2：CUDA wheel 不打包运行时 DLL，不在 PATH 会致 llama.dll 加载失败、GPU offload 静默回退 CPU → 前置注入 nvidia/*/bin 到 PATH；
- 预编译 **wheel 静默 CPU-only** 坑：PyPI 预编译 wheel 是 CPU-only，直接 pip install 会用 wheel 忽略 **CMAKE_ARGS**，导致 GPU offload 静默失效 → --no-binary llama-cpp-python --force-reinstall 强制源码编译 + FORCE_CMAKE=1，编译后二次校验 llama_supports_gpu_offload()。

4.6.3 依赖冲突定位与固定

semgrep 严格锁 opentelemetry ~1.37.0，chromadb 的 grpc exporter 被升到 1.44.0，common/proto/sdk 被“撕成两半” → ModuleNotFoundError。修复：全部 otel 包钉回同一版本，并写“检测家族版本 >1 个不同主版本即判定撕裂”的自动对齐。经典“两个依赖各自合法、合起来崩”场景。

4.6.4 “存在但损坏”的坏依赖检测

PATH 可见 ≠ 可运行：安全工具安装区分“缺失”与“存在但损坏”（实际跑 --version 冒烟）；应用级运行时依赖用子进程 import 冒烟（避免污染本进程 importlib）；修复后复查冒烟。

4.6.5 CI 的“永不阻断 PR”设计哲学

LLM 扫描作为“可选信息”而非硬门槛——Ollama 不可用、模型缺失、单文件出错都只输出警告并退出码 0。三处可写：① 分层依赖检查（核心层不得反向依赖 app 层，否则阻断）；② 离线模式（CI 无 HF 缓存，Scanner 自动回退纯 LLM 模式）；③ 模型拉取失败自动回退。

4.6.6 已识别取舍与设计边界（如实声明）

本节集中记录设计中明知代价而选择的部分，供评审对照，避免把取舍当成疏漏：

取舍	选择	代价	理由
共形预测的启用状态	未校准时不启用(当前生产即为未校准)	放弃一层统计门控	校准文件来源不满足可交换性，“覆盖率保证”不成立；不宣称不成立的保证优先于多一层门控
复核预算分配	无候选文件按风险分 / 盲区分档，零风险文件不送模型	这部分文件的结果是“未复核”而非“已安全”	推理预算有限；把“没看过”如实标注,比让每个文件都过一次模型更诚实也更可行
规则级误报治理	只降权、不剔除	误报规则仍会产生候选、仍消耗裁决预算	误报取决于文件而非规则，按规则“见到就跳过”会制造样本级静默盲区(有历史事故)
进程内推理	单进程单工作线程串行	批量扫描按文件数线性增长,无并发吞吐	显存约束下的确定性与可诊断性优先;高吞吐场景另设 vLLM 端点
全票门槛与误报	保留全票判定的回填资格,但叠加与模型无关的证据门	需要更多门控代码	全票刻画的是“模型很自信”而非“模型很可能对”，单独使用与已知失效模式正交

取舍	选择	代价	理由
跨文件数据流	单文件为主，跨文件攻击路径为弱项	复杂跨文件漏洞漏报	单文件分析无需构建跨文件索引，可保持离线、可解释；跨文件属后续工作

4.6.7 设计沿革（可选章节）

两阶段架构的最初设计书完成于架构上线之前，其中“反思 LLM 定位”的设计动机与方案对比仍有参考价值。引用时只取设计动机与方案对比，不引其状态描述与数字（该文档的状态行已过时）。

更完整的技术选型调研（反事实验证、共形预测）见技术选型调研记录，仅作设计依据，非数据来源。

第五篇 数据存储设计

本篇对应官方材料：数据库设计说明书（如必要） 说明：本系统无独立数据库（不依赖 MySQL / PostgreSQL / SQLite 等关系型数据库服务）。系统运行所依赖的全部数据以本地文件 + 向量库 + 浏览器本地存储三类形式持久化。本篇按官方「如必要」的口径，如实描述本地数据存储设计。

5.1 总体说明

系统的数据设计目标是单机自持、离线可用、隐私不出本机。因此刻意避免了数据库服务依赖，采用三类本地持久化形式：

类型	载体	用途	是否入库分发
文件存储	models/ 下各类 JSON / YAML / 模型权重	模型注册表、校准参数、抑制信号、规则文件、adapter	规则文件入库存档；权重按形态分发
向量知识库	data/chroma_db/	漏洞知识检索 (RAG)	否（本地构建）
浏览器本地存储	localStorage / sessionStorage	扫描历史、报告记录、主题偏好	否（客户端）

设计原则：

- 1. 确定性工具与模型信号的文件分层——确定性规则文件与模型回填信号分开存放，抑制过滤只作用于裁决档，确定性工具自判不受模型意见影响；
- 2. 运行期不联网——HuggingFace、Semgrep 规则全部本地化，HF_HUB_OFFLINE=1 + TRANSFORMERS_OFFLINE=1；
- 3. 可卸载——所有运行期数据都在项目目录内，卸载脚本可按范围清理（见 §5.5）。

5.2 文件存储

5.2.1 数据目录总览

```
自持数据/
├─ models/
│   ├── signal_registry.json          # 工具-模型闭环：规则负向记忆 / 排序加权 / 类型校正
│   ├── conformal_calibration*.json  # 共形预测校准阈值（生产当前未加载，见 §5.2.3）
│   ├── semgrep_rules/               # 本地化 Semgrep registry 规则（离线可用）
│   ├── adapter_*/                   # LoRA 适配器（按 stage 分目录存放）
│   ├── ollama/                      # Ollama 模型存储（启动器锁定到此目录）
│   └─ <GGUF 基座>.gguf              # LlamaCPP 后端基座（用户自备）
├─ data/
│   └─ chroma_db/                    # 向量知识库（见 §5.3）
└─ logs/                             # 运行日志
```

5.2.2 信号注册表 `signal_registry.json`

用途：记录模型对每条工具规则的判定质量信号，是本系统“工具-模型自适应闭环”的持久化载体。

字段（语义）	说明
规则计数	按 (rule_id, taint_type) 累计的确认/否定计数
confirmed_files / rejected_files	独立文件级去重集合（跨样本聚合门槛的基础）
负向记忆	该规则在哪些文件上被高置信否定——读端据此降低该文件上该规则的候选排序权重，不删除候选
corrected_type_counts	按 (file, corrected_type) 去重的类型校正计数
待学习池	“工具漏召但模型全票判中”的代码特征，须经独立验证集复验才转正

读写特性：

- 写入时机：在最终结论确定之后写入（聚合完成、三道门控均已生效），以最终采信结论而非模型原始投票为写入依据；
- 写入后自动 `save()`，采用原子写（临时文件 + `os.replace()`）；
- 读入门槛（五重）：读端按第四篇 §4.3.4 的五重门槛逐项校验；未达门槛保留原值（防止模型错误判定污染确定性工具），通过门槛的负向记忆也只降权、不剔除——候选集合始终由确定性规则产出；
- 评估隔离：评估时以 `--no-signal-feedback` 关闭回填，避免“测量与自举”相互污染（历史事故：跨轮次累积的负向记忆导致工具召回系统性崩塌）。

5.2.3 共形校准文件与其当前状态

用途：存放共形预测的标签条件分位数阈值，把“评估侧拟合 → 生产端加载”打通。

字段（语义）	说明
alpha	显著性水平，默认 0.1（目标覆盖率 90%）
标签条件分位数	漏洞 / 安全 两个标签各自的非一致性分位数 q
校准元信息	校准集规模、生成时间

契约约束：

- 校准数据必须带真实标签（每条为完整 N 采样投票向量 + label）；
- 禁止 **in-sample** 校准——校准集必须与生产分布可交换，否则覆盖率保证不成立；
- 加载失败安全降级——文件缺失或解析失败时返回未校准状态，预测全部落入“不确定”并回退旧投票逻辑，不中断扫描。

当前状态：此前生产端自动加载的校准文件为 **in-sample** 拟合，不满足可交换性前提；已移出自动加载路径并存档（文件名带 `in_sample_leaked` 标识以免误用），生产端因此处于未校准状

态——Layer 1 不参与判定，结论交由其余门控与投票逻辑。发现经过与取舍见第四篇 §4.3.2；需要启用时，应以独立开发集重新校准并显式指定校准源。

5.2.4 规则文件 `models/semgrep_rules/`

Semgrep registry 规则包（`p/security-audit`、`p/owasp-top-ten`）本地化为纯文本 YAML 并随软件入库，版本固定。

项目	说明
更新方式	<code>python tools/fetch_semgrep_rules.py</code> （幂等，已存在则跳过；需联网）
校验方式	<code>python tools/fetch_semgrep_rules.py --check</code>
缺失降级	规则文件缺失时，工具层自动降级为在线拉取（并提示）

另有一组自研 **Semgrep** 召回规则位于核心包内（`taint` 四件 `sql_i_taint` / `cmd_i_taint` / `xss_taint` / `code_i_taint`，结构化补位两件 `go_cmd_i_shell` / `c_cmd_i_system`），与上述官方 registry 规则是两个目录两套规则，用途不同：前者为自研整文件召回（含工具缺口补位），后者为通用安全审计规则集。

5.2.5 模型权重与适配器

形态	存储	尺寸
发布模型 v9max（Ollama）	<code>models/ollama/</code> （启动器锁定）	约 4.7 GB（Q4_K_M GGUF）
LoRA 适配器	<code>models/adapter_*</code> / （含 <code>adapter_model.safetensors</code> ）	数十 MB 级
基座（Transformers 形态）	HuggingFace 本地缓存（ <code>models/transformers/.hf_home</code> ，启动器经 <code>HF_HOME</code> 锁定）	数 GB（NF4 量化后约 4.5 GB）

adapter 定位规则（分层探测）：主位 `models/adapter*` → 兜底训练产物目录；打分时上线物 **stage2** 优先于 **stage1** / 历史 **checkpoint**，保证上线版本不被训练轮次产物顶替。

5.2.6 Schema 与训练口径一致性

输出 **schema** 在核心层与两个推理客户端中定义，三处必须同步。历史问题：**schema** 定义了某字段但训练标签中 99% 不使用，导致模型“按 **schema** 修正输出”引发格式不匹配——三处 **schema** 必须同步是本系统的数据一致性约束。

5.3 向量知识库

5.3.1 存储形态

项目	选择
向量库	Chroma（本地持久化目录 data/chroma_db/）
embedding 模型	BAAI/bge-m3（多语言，100+ 语言）
集合名	vulnerability_knowledge
检索参数	查询文本取源码前 2000 字符；n_results = 3

5.3.2 设计依据

- 选型 bge-m3 而非英文小模型：知识库主要是中文漏洞资料，英文专用模型的向量化质量不足；
- 强制离线：运行时设置 HF_HUB_OFFLINE=1 + TRANSFORMERS_OFFLINE=1，绝不联网下载；
- 三级模型路径解析：环境变量 CHROMA_EMBEDDING_MODEL_PATH → HuggingFace 本地缓存自动发现（按快照目录命名规则搜索 + 配置校验）→ 明确错误提示；
- 延迟初始化与优雅降级：Chroma 首次使用时才初始化，初始化失败静默降级为纯 LLM 模式并记忆失败（避免反复重试）——这是“RAG 增加 <2% 推理时间”的工程依据。

5.3.3 构建与重建

向量库不随软件分发（体积大且可由知识源重建），系统附带知识库构建脚本。首次使用或需要重建时（如更换 embedding 模型后），运行软件包内的构建脚本即可：

```
# 在软件根目录执行：从随附的知识源重建向量库（会覆盖旧库）
python <随附脚本>/build_knowledge.py
```

触发场景：

场景	是否需要重建
首次启用 RAG	需要（本机尚无向量库）
日常扫描	不需要（库已就绪，直接加载）
更换 embedding 模型	需要（旧库向量与新模型不兼容）
检索结果明显异常	需排查，必要时重建

构建过程需要本地已缓存 embedding 模型（见 §5.3.2 三级路径解析）；构建完成后运行时强制离线，不再联网。

5.4 前端持久化

5.4.1 localStorage 用途

键（语义）	内容	说明
扫描轮次	本轮扫描的文件与结果	页面刷新不丢失
扫描结果	历史结果缓存	供报告历史页读取
报告历史	已生成报告的索引	—
主题偏好	深/浅色主题设置	<head> 内联脚本在 CSS 加载前读取，规避闪烁
模型管理状态	当前激活模型等	—

5.4.2 设计要点

- 双保险入账：批量 NDJSON 流式结果以双保险保证“一轮只入账一次”；
- 容量超限的降级策略：结果对象携带完整源码，大批量扫描后序列化体积可能超过浏览器存储配额。写入失败时不静默丢弃：先剥离体积大头（原始代码、模型原始输出）降级重试，仍失败才告警——保证“历史与报告继续可用”是默认行为，而不是配额一满就整体失效；
- 旧数据自动迁移——数据结构演进后，读取时兼容旧格式并升级；
- **sessionStorage** 补充：深度健康检查结果（最坏 30s+）用 sessionStorage 缓存 + AbortController 在页面切换时主动中止；
- 无服务端用户数据——浏览器本地存储是纯客户端数据，清除浏览器数据即清空，不影响系统运行。

5.5 数据生命周期

5.5.1 运行期数据产生与销毁

数据	产生	销毁
GitHub 仓库副本	浅克隆到系统临时目录	扫描后 <code>shutil.rmtree</code> 立即删除（120 秒超时保护）
URL 抓取内容	内存流式读取（超 2 MB 截断）	不落盘
批量扫描结果	内存 + 浏览器 localStorage	用户清除浏览器数据
信号注册表	每次裁决后原子写入	卸载脚本或手动删除
共形校准文件	评估侧拟合后导出（生产当前未加载）	卸载脚本或手动删除

5.5.2 网络访问边界

阶段	是否联网
首次安装	需要（下载依赖 + 模型）
日常运行	不联网（HF 强制离线、Semgrep 规则本地化）

阶段	是否联网
GitHub 仓库扫描	需要（克隆目标仓库，仅此功能）
URL 抓取扫描	需要（抓取目标页面，仅此功能）

5.5.3 卸载清理范围

从数据生命周期视角与卸载相关的三类落盘：本地运行数据（data/chroma_db、outputs/、logs/、各类缓存等，随卸载删除）、全局目录（~/.ollama 与 ~/.cache/huggingface——卸载会删除其中所有模型/缓存，包括其他项目用到的）、以及不会被触碰的外部共享工具（bandit / semgrep 等）。完整的八项清理清单、保留项与注意事项见第八篇 §8.7.3~§8.7.5。

5.5.4 数据可信度保障机制

系统的数据设计中有三条与“数据可信度”直接相关的机制，属于存储层的制度性约束：

机制	内容
冻结集锁定	关键测试集用 SHA-256 哈希锁定，强制“冻结集永不入训练轮”
泄漏审计	训练/测试文件按 3 行滑动窗口计算 Jaccard 相似度，>30% 视为高重叠直接删除，>15% 人工复核
评估隔离	评估环境与自举机制隔离(关闭信号回填、禁用 in-sample 校准加载)——“测量”与“自举”必须分离

第六篇 软件界面设计书

本篇对应官方材料: 软件界面设计书 截图说明: 本篇全部插图均为实机截图 (Web 界面 8 张 + VS Code 插件 1 张 + 模型管理与主题对照 2 张); 截图中的品牌位与模型完整名称已按匿名要求统一隐去; 部分截图中保留的红色框线为素材要点标注 (非界面元素)。IntelliJ 插件因未实机安装, 以文字说明其功能对等性 (见 §6.3.2)。

6.1 设计系统

6.1.1 设计哲学

“专业的安全工具应该是隐形的, 数据 > 装饰”

不以动画堆砌博眼球, 而是让安全工具“不打扰用户”, 把数据清晰度放在首位。

6.1.2 双层令牌体系

层	文件	内容
基础令牌	design-tokens.css	全站 CSS 变量: 颜色 / 圆角 / 间距 / 阴影 / 字体
语义令牌	@theme inline 映射	映射到框架语义 token, 5 个页面共用
动效系统	lively.css	统一动效曲线、玻璃卡片、导航栏与健康指示灯

特性:

- 深浅双色系完整覆盖;
- <head> 内联同步脚本在 CSS 加载前读取主题设置并应用主题类, 规避闪烁 (FOUC);
- 支持 prefers-reduced-motion (尊重系统的“减少动效”设置)。

6.1.3 零依赖手写 SVG 图表

全部图表手写 SVG, 无第三方图表库:

图表	实现
折线趋势图	<polyline> + 渐变
环形分布	<circle> + stroke-dasharray
柱状图	<rect> 系列
评分圆环	用三角函数逐帧重算弧路径的 d 属性, 从根上规避特定浏览器“圆头线帽 + dasharray 动画”的渲染位移缺陷

收益: 零外部依赖、离线可用、体积可控、渲染一致性可保证。

6.1.4 视觉规范摘要

元素	规范
主色	沿用安全工具惯用的冷色调，配合语义色（危险/警告/安全）
风险等级色	Critical / High / Medium / Low 四级固定语义色
卡片	玻璃拟态（半透明 + 模糊），深浅主题各自适配
健康指示灯	引擎状态用颜色 + 文案双通道表达（不只依赖颜色，兼顾可访问性）
圆角 / 间距	由令牌统一，避免页面间漂移

6.2 信息架构与页面清单

6.2.1 页面清单

页面	路径	功能定位	主要元素
欢迎页	/welcome.html	首次访问引导	系统定位说明、入口导航
仪表盘	/index.html	总览	扫描历史统计、引擎健康状态、最近发现漏洞、评分圆环、趋势图
扫描工作台	/scan.html	核心功能页	四种输入方式 Tab、高级选项、结果区（含来源筛选 chip）、报告下载、模型管理抽屉
CWE 样本库	/cwe.html	知识查阅	25 类常见 CWE 的示例与修复方案（可搜索）
安全态势	/posture.html	态势总览	整体安全态势统计与趋势

6.2.2 导航结构

- 顶部导航栏常驻（含页面切换、主题切换、引擎健康指示灯）；
- 扫描工作台内部用 Tab 切换四种输入方式；
- 页面间支持深链接跳转（演示样本库 → 扫描工作台 → CWE 详情）。

6.2.3 共享资源

文件	作用
nivis-common.js	全站共享逻辑（模型管理、结果渲染、请求封装）
backend-badge.js	引擎健康状态指示灯
theme.js	主题切换与跨标签页同步
design-tokens.css / lively.css	视觉令牌与动效

6.3 界面截图

截取要求：分辨率满足打印可读（等效 ≥150 DPI）；界面文字与全书命名口径一致；截图内的模型完整名称、品牌位与本机路径须先行隐去（模型命名空间等同于个人信息）。

6.3.1 Web 界面

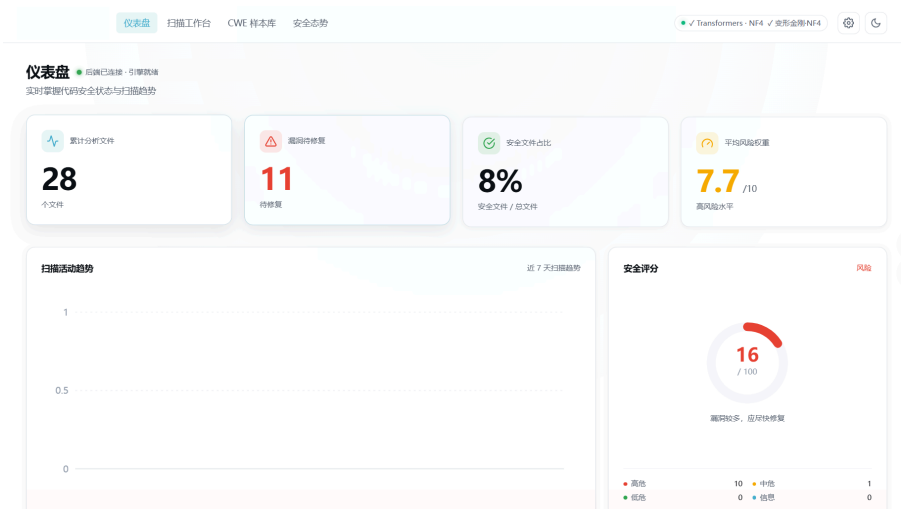


图 6-1 仪表盘（扫描历史统计、引擎健康状态、评分圆环与漏洞趋势；图示为全新安装后的初始统计态，「扫描活动趋势」暂无历史数据，评分圆环为百分制安全评分、与指标卡的 0~10 风险权重分属不同量纲）

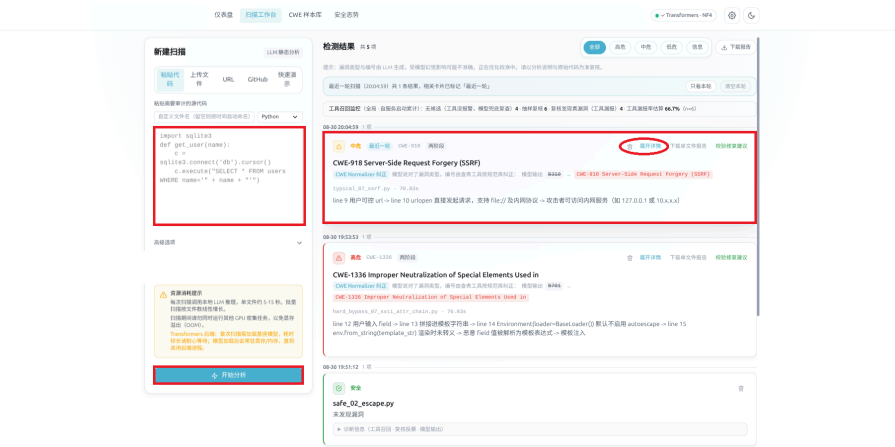


图 6-2 扫描工作台（四种输入方式 Tab、高级选项与结果区）

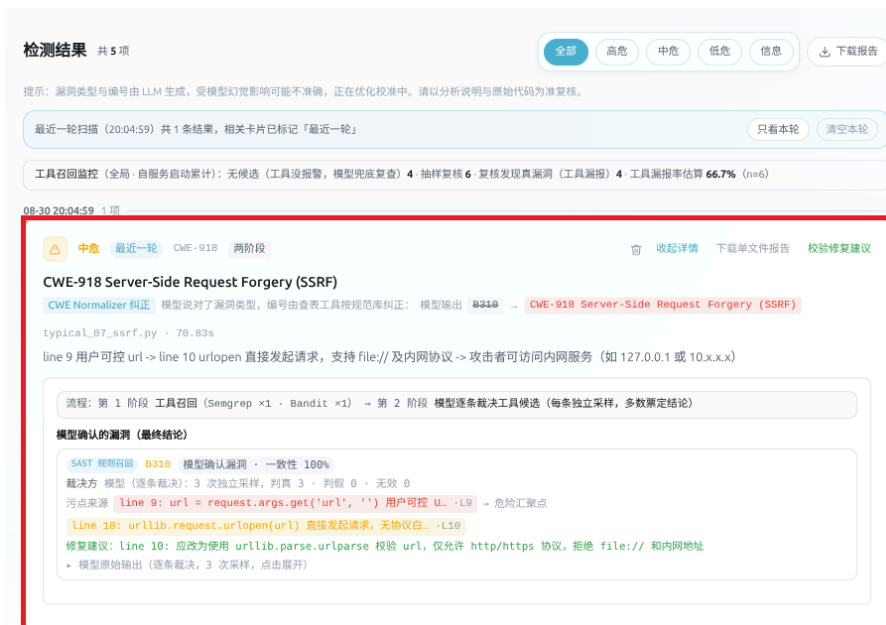


图 6-3 扫描结果详情（漏洞卡片、CWE 标签与修复建议）

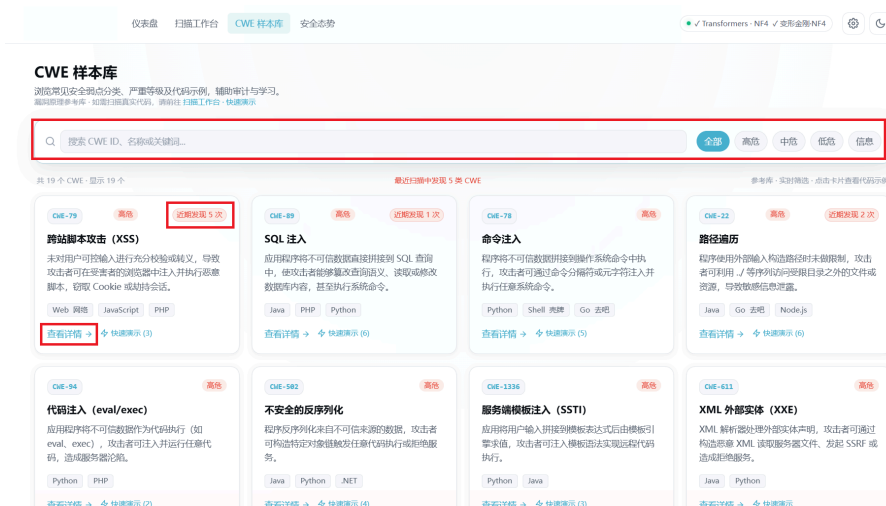


图 6-4 CWE 样本库（25 类常见弱点的可搜索示例与修复方案）

CWE-79

高危 · 注入类

×

跨站脚本攻击 (XSS)

描述

未对用户可控输入进行充分校验或转义，导致攻击者可在受害者的浏览器中注入并执行恶意脚本，窃取 Cookie 或劫持会话。

检测要点

检测要点：用户可控数据未经 HTML 实体编码就进入 HTML 输出；使用 innerHTML/document.write 接收外部数据；DOM 直接拼接 location.hash、document.referrer 等危险源。

漏洞代码示例 javascript

```
// 漏洞代码：直接写入 innerHTML
document.getElementById("out").innerHTML =
    location.hash.slice(1);
```

安全代码示例 javascript

```
// 安全代码：使用 textContent
document.getElementById("out").textContent =
    location.hash.slice(1);
```

修复建议

修复建议：对所有用户输入进行 HTML 转义 (html.escape / htmlspecialchars)；使用 textContent 代替 innerHTML；实施严格的 CSP 策略；富文本使用白名单过滤库 (如 DOMPurify)。

受影响语言 / 技术栈

Web

JavaScript

PHP

图 6-5 CWE 详情（示例代码与对应修复方案）

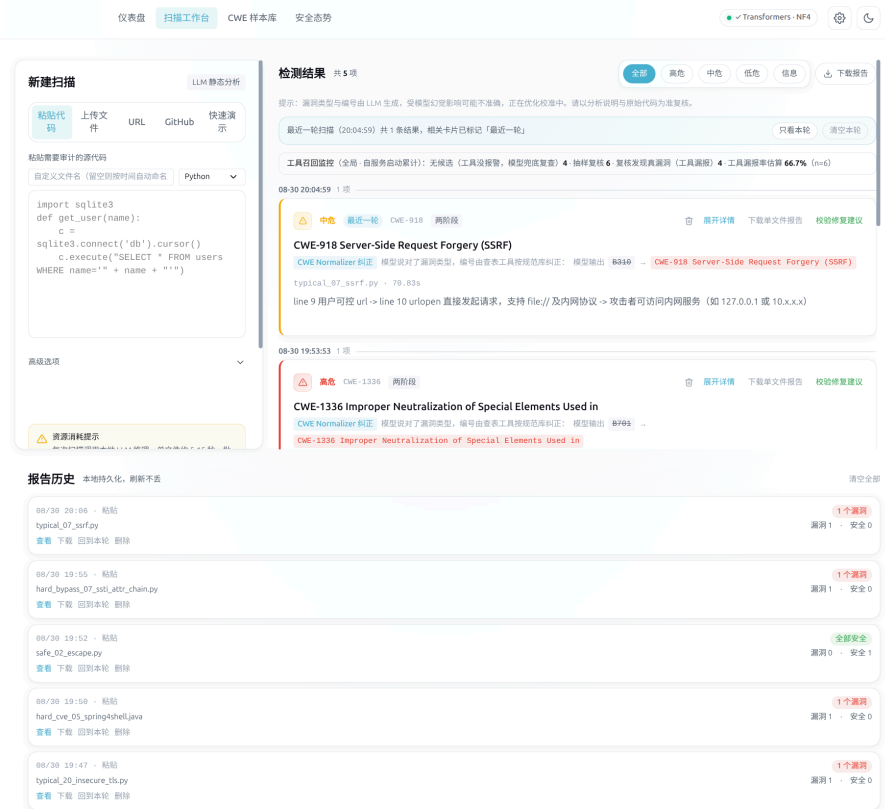


图 6-6 报告历史（历次扫描报告索引）

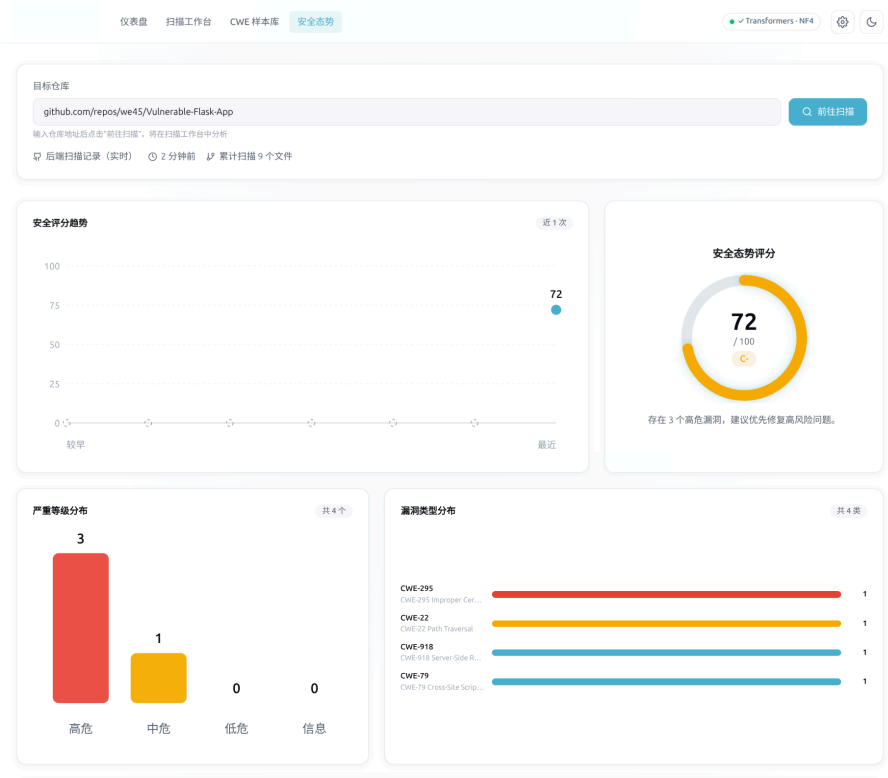


图 6-7a 安全态势（上半）（安全评分趋势折线 + 安全态势评分圆环 + 严重等级分布柱状图 + 漏洞类型分布条形图）



图 6-7b 安全态势（下半）（高危文件清单 + 扫描概览统计 + 优先修复清单表：按 P1~P4 优先级排列 CWE 与文件路径）

6.3.2 编辑器插件

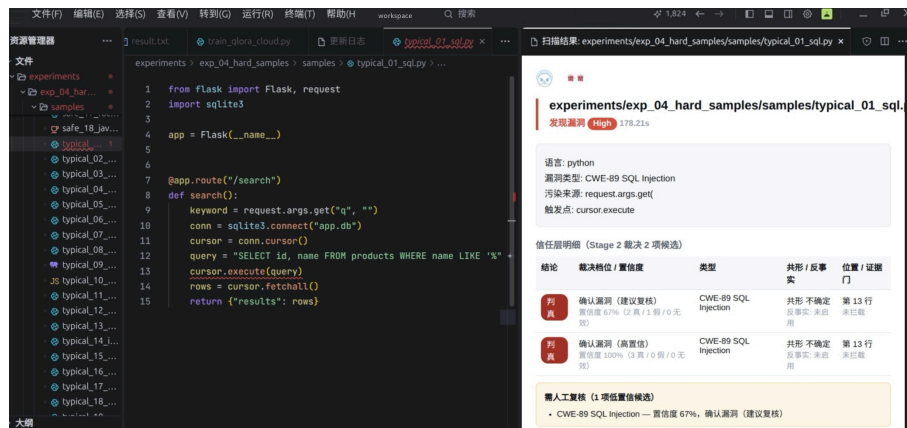


图 6-8 VS Code 插件：行级诊断与信任层明细（左侧为编辑器中的待审计文件；右侧结果面板自上而下为：判定与风险等级、语言、CWE 类型、污染来源与触发点行号，以及信任层明细表——逐条候选列出裁决档位与一致性置信度、CWE 类型、共形预测与反事实验证状态、证据门是否拦截、行号定位）

IntelliJ IDEA 插件提供与之同构的能力：编辑器内选中代码（不选则扫整个文件）→ 右键菜单或快捷键触发 → 结果以 IDE 气球通知展示结论、修复建议与信任层摘要，后端地址可在弹窗中修改。两个插件共用同一后端接口与同一套结果语义，只是展示载体不同（Webview 面板 / 原生通知）；因本机未安装该 IDE，此处不作截图，功能对照见第二篇 §2.5。

6.3.3 设置与模型管理



图 6-9 模型管理（当前活动模型、已安装模型列表、可拉取模型列表与「拉取」入口；进程内后端形态下该区块如实提示“本形态仅随本地 LoRA adapter 分发”）

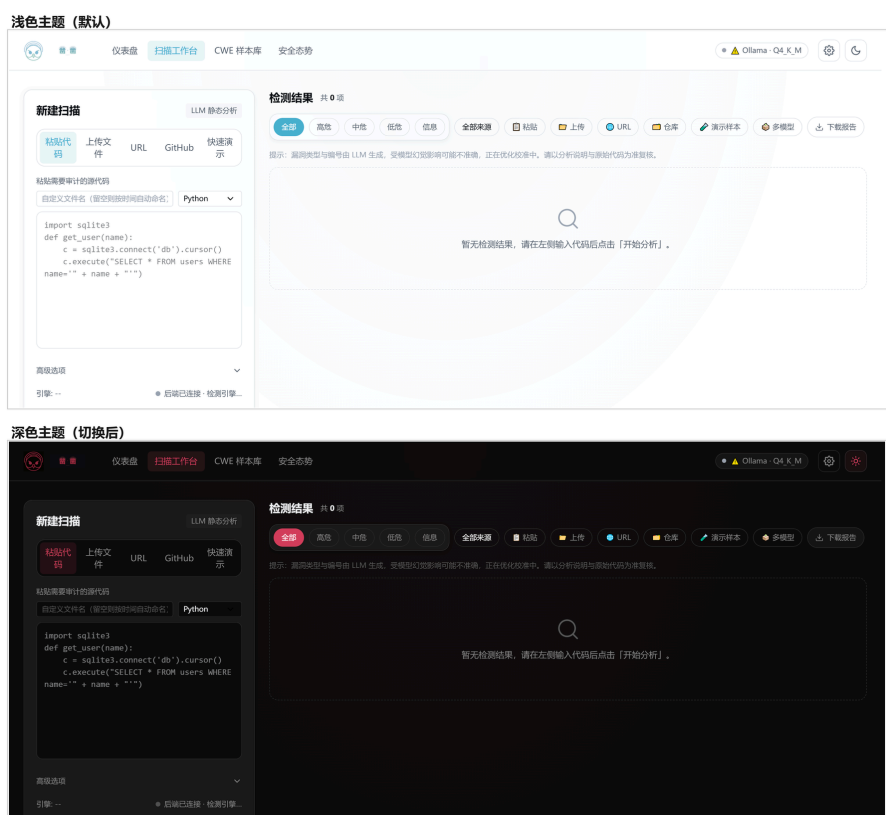


图 6-10 深浅主题对比（同一页面、同一滚动位置的浅色与深色主题；主题切换采用视图过渡 API 做交叉淡化，本图为静态对照）

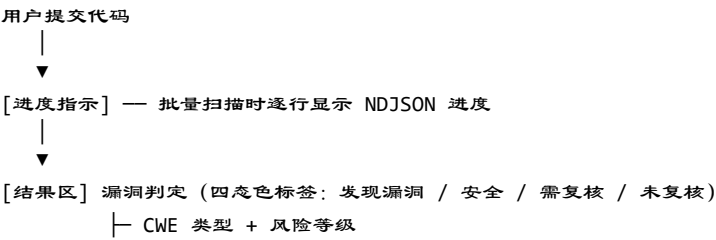
设置抽屉同时给出当前推理后端的精度形态报告（基座量化方式、LoRA 是否被二次量化、运行设备），使用户能直接看到“为什么两档形态精度不同”，而不必翻阅文档。

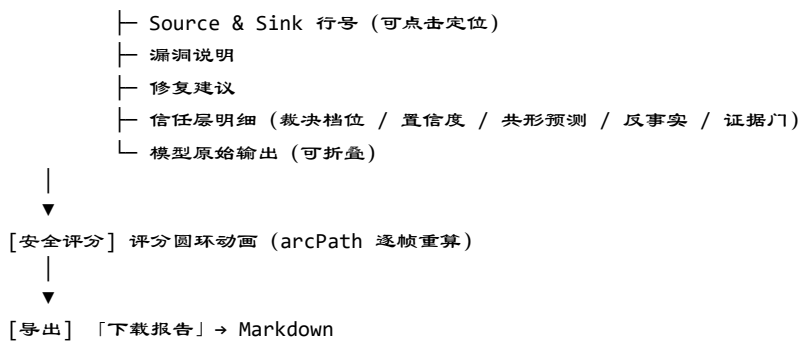
6. 4 关键交互

6. 4. 1 主题切换

采用 View Transition API 实现交叉淡化，带 `_inTransition` 防重入保护，支持跨标签页同步；不支持该 API 的浏览器直接切换（优雅降级）。

6. 4. 2 扫描结果呈现流程





6.4.3 交互健壮性设计

场景	设计
深度健康检查耗时长（最坏 30s+）	sessionStorage 缓存 + AbortController 在页面切换时主动中止，避免占满同域 HTTP 并发连接
批量结果重复入账	双保险机制保证“一轮只入账一次”
数据格式演进	旧数据自动迁移
长连接被代理掐断	模型下载流空闲时发心跳帧保活；响应头禁用中间层缓冲
评分语义	统一扣分制，规则与“需复核 / 未复核”单列口径见第二篇 §2.3.3
深链接	演示样本库支持多类查询参数深链接，页面间互相跳转闭环

6.4.4 可访问性

- 风险等级与引擎状态用颜色 + 文案双通道表达，不单纯依赖颜色；
- 尊重系统 prefers-reduced-motion；
- 表单与按钮保持足够的点击区域与键盘可达性。

6.4.5 编辑器插件交互

插件	交互
VS Code	右键菜单 / 命令面板 / 编辑器标题栏图标三种入口；结果以 Webview 面板展示，含信任层明细表格；每个确认的漏洞按行号标红色波浪线；保存时自动扫描采用 onDidSave（而非保存前阻塞），避免等扫描完成才落盘
IntelliJ	编辑器内选中代码 → 右键菜单或快捷键；结果以气球通知弹出；后端地址支持在弹窗中修改

第七篇 用户操作手册

本篇对应官方材料：用户操作手册 配图说明：本篇插图（图 7-1~7-6）均为系统实机截图，已按匿名要求隐去品牌位、模型全名与本机路径。

7.1 三个最常用的任务（5 分钟上手）

7.1.1 任务 A：扫描一段代码或一个文件

- 1. 打开 `http://localhost:8765` → 进入扫描工作台；
- 2. 「粘贴代码」标签 → 选语言 → 粘贴代码 → 点「开始分析」；或者「上传文件」标签 → 拖入一个或多个代码文件；
- 3. 等待十几秒（单文件典型 5~15 秒，长文件更久），结果区给出：是否漏洞、CWE 类型、风险等级、漏洞原理说明、修复建议。

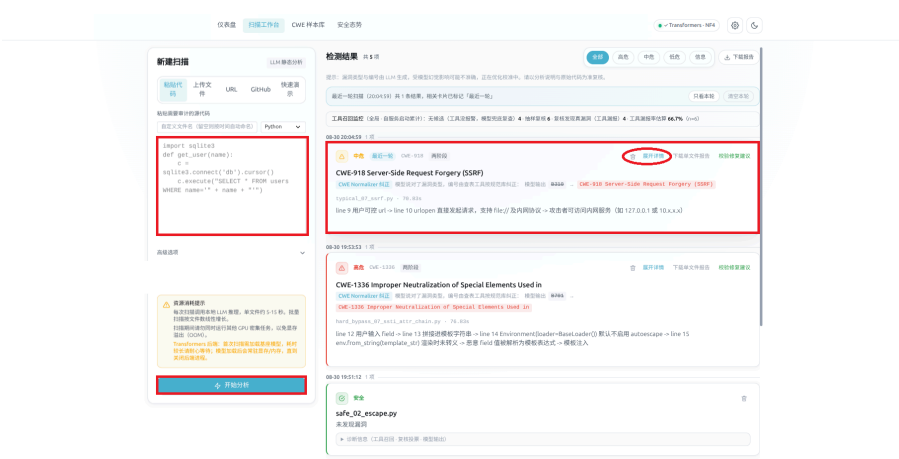


图 7-1 扫描工作台（四种输入方式标签页、高级选项与结果区；任务 A/B/C 均在本页完成）

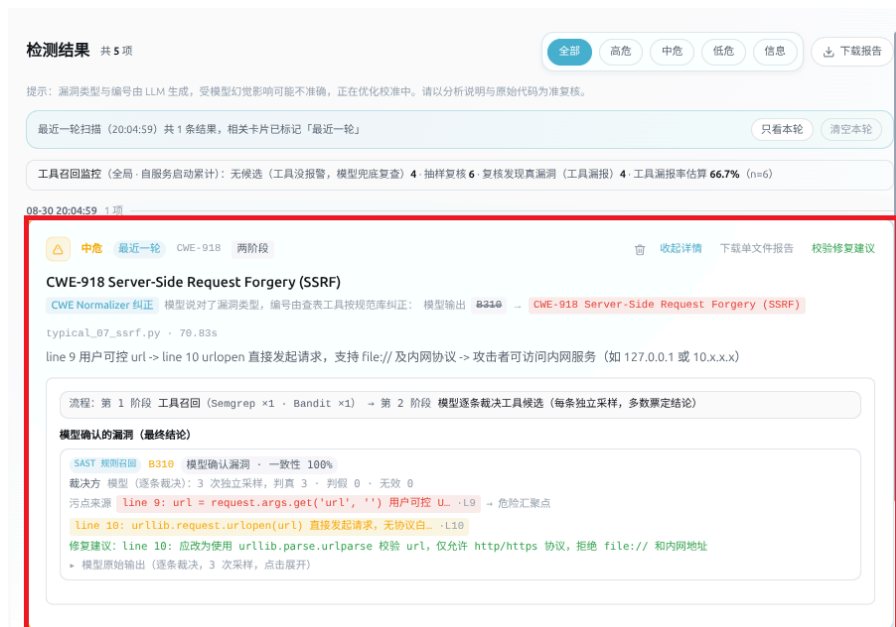


图 7-2 扫描结果详情（判定标签、CWE 类型与风险等级、漏洞说明与行号锚定的修复建议）

7.1.2 任务 B：批量扫描一个项目并导出报告

1. 扫描工作台 → 「上传文件」 → 一次拖入整个模块的代码文件（最多 200 个）；
2. 点「开始分析」，页面实时显示进度（扫完一个出一个）；
3. 完成后点「下载报告」，得到一份 Markdown 报告：漏洞汇总表 + 每个文件的详细结果 + 修复建议。

7.1.3 任务 C：扫描 GitHub 仓库或网页

- 「GitHub」标签 → 粘贴仓库地址 → 设置最多扫描文件数（网页端默认 20，大仓库建议自己限定） → 开始。系统浅克隆仓库（只读、120 秒超时、扫完即删，不会在用户机器上留下仓库副本）；
- 「URL」标签 → 输入网页地址 → 系统抓取页面里的脚本代码逐个分析。

7.2 读懂扫描结果

7.2.1 四种判定，各有含义

判定	含义	应当做什么
发现漏洞	系统确信存在漏洞（模型多次采样一致 + 通过证据校验）	按修复建议处理

判定	含义	应当做什么
安全	已实际复核且未发现问题（工具没召回可疑点，且该文件确实进入了复核）	常规流程即可
需人工复核	系统有怀疑但证据不足，故意不替用户下结论	花一分钟看一眼标注位置
未复核	工具零召回、且本轮注意力预算没有覆盖到该文件（LLM 从未被调用）	按需人工查看；不要把它当作“已确认安全”

「需人工复核」不是出错，是设计：本系统的原则是“宁可多一道人工复核，也不漏报真漏洞、不错杀不确定的判定”。复核结果会单列，不计入安全评分。

「未复核」同理：该文件本轮未被分配分析预算、从未进入分析——保留“未复核”这一状态，正是为了让“没有看过”如实可见，而不是被界面渲染成“安全”。

7.2.2 每条结果包含什么

- **CWE 类型**：漏洞编号与名称（如 CWE-89 SQL 注入）；
- **风险等级**：Critical / High / Medium / Low（影响安全评分扣分）；
- **Source & Sink**：污点数据的来源行与触发行（有数据流的漏洞才显示），配合编辑器插件可直达代码行；
- **漏洞说明**：自然语言解释“为什么这是漏洞”；
- **修复建议**：行号锚定的局部修复建议；
- **信任层明细**：裁决档位、一致性置信度、共形预测、反事实验证与证据门；低置信候选列入人工复核清单；
- **模型原始输出**：可折叠查看，用于核对。

7.2.3 安全评分

系统按 Critical 每项 -8 / High -8 / Medium -4 / Low -2 / Info -0.5 累计扣分给出安全评分；“需人工复核”的样本单列展示，不参与扣分。

7.2.4 为什么结果可能和 Bandit / Semgrep 不一样

三个常见原因，都属正常：

1. 它把“安全写法”放行了：参数化查询、subprocess 列表参数、shlex.quote 等有效防御会被识别，不再像规则工具那样照报误报；
2. 它报了规则工具不报的：需要理解数据流 / 业务语义的漏洞（认证缺失、信任边界绕过等），规则工具天然覆盖不到；
3. 它给出“需人工复核”而不是硬判：低置信判定会被送人工复核，而不是随机选边。

7.3 Web 界面导航

页面	干什么用
欢迎页	首次访问的引导与入口导航
仪表盘	历史扫描统计、引擎健康状态、最近发现的漏洞
扫描工作台	核心功能页（§7.1 的任务都在这里做）
CWE 样本库	25 类常见漏洞的可搜索示例与修复方案——看不懂某个 CWE 时来这里
安全态势	整体安全态势总览

界面支持深浅色主题切换；所有图表为手写 SVG 实现，无外部图表库，离线可用。

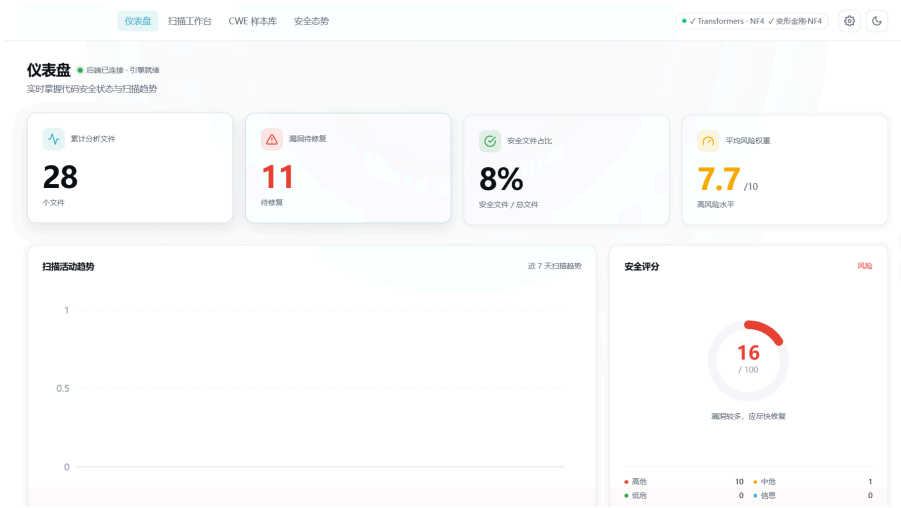


图 7-3 仪表盘（历史扫描统计、引擎健康状态、评分圆环与最近发现的漏洞）

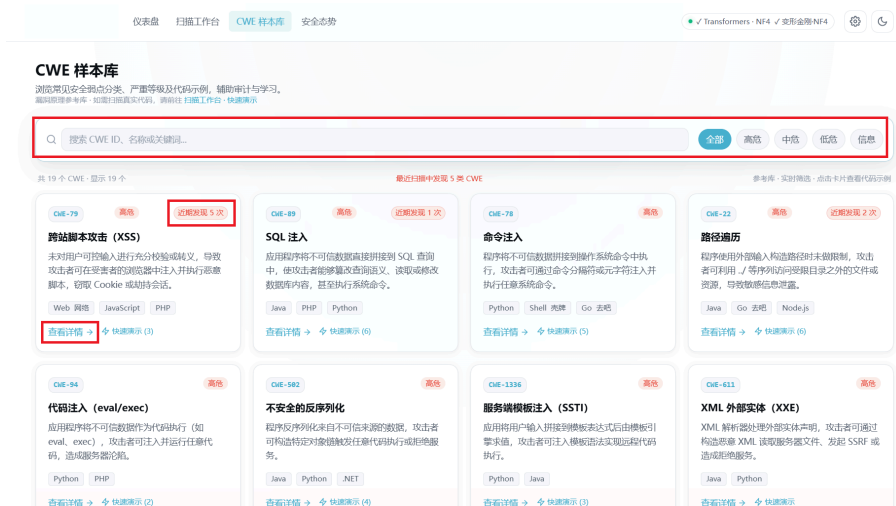


图 7-4 CWE 样本库（25 类常见漏洞的可搜索示例与修复方案，可从扫描结果深链接直达对应类型）

7.3.1 扫描工作台的四种输入方式

输入方式	操作
粘贴代码	选语言下拉框 → 粘贴源码 → 「开始分析」
上传文件	拖拽到上传区或点击选择（支持多文件）→ 「开始分析」→ 批量时显示实时进度 → 完成后可「下载报告」
URL 扫描	输入目标站点 URL → 「开始分析」→ 自动抓取页面脚本逐个扫描，结果显示页面标题、发现脚本数、各脚本分析
GitHub 仓库	输入仓库地址 → 设置最大扫描文件数（默认 20）→ 「开始分析」→ 浅克隆 → 遍历 → 批量扫描 → 显示汇总

7.3.2 高级选项

在扫描工作台点击「高级选项」可展开：

选项	说明
多模型投票	勾选后选择 ≥ 2 个模型，系统顺序加载各模型并投票聚合结果（更准但更慢）
两阶段扫描	系统唯一扫描路径（工具召回 + LLM 自一致率裁决），无需勾选即生效；本开关仅控制下方采样数设置是否可调
自一致率采样数	两阶段 LLM 裁决的采样次数（1~10，默认 3）。采样越多越准、越慢

7.3.3 API 文档

后端提供完整的 REST API，访问 <http://localhost:8765/docs> 查看 Swagger 交互文档。

调用示例（单文件分析）：

```
curl -X POST http://localhost:8765/api/analyze \
-H "Content-Type: application/json" \
-d '{
  "code": "import sqlite3\ndef get_user(name):\n    c = sqlite3.connect(\"db\").cursor()\n    c.execute(\"SELECT * FROM users WHERE name=\'\'\'\" + name + \'\'\'\'\'\"),\n  "language": "python",\n  "filename": "vuln.py"\n}'
```

7.4 输入限制与支持范围

支持 7 种代码语言及对应文件类型；单段代码最多 200 万字符、单文件最大 2 MB、单次批量最多 200 个文件（总计 10 MB）、GitHub 扫描默认 20 个文件（上限 500）。完整限制表见附录 C；超限会被前端直接拦截提示（后端再做一道同样的校验，返回 413），不会白等一场。

7.5 高级功能

扫描工作台「高级选项」里：

「自一致率采样数」与「多模型投票」两张开关的作用机制与建议见 §7.3.2；这里补充第三个：

功能	说明	建议
RAG 知识库	检索内置的漏洞知识注入判定过程，增强可解释性	默认关；开启后推理时间增加 <2%

注：Bandit / Semgrep / Gitleaks / Trivy 等外部工具已内置于两阶段管线的召回层（见 §7.3.2「两阶段扫描」），无需单独安装或勾选。

7.6 在编辑器里用（VS Code / IntelliJ 插件）

前置：后端服务已启动并保持运行（插件只是“遥控”后端，后端关了插件就不能用）。

7.6.1 VS Code

- 1. 安装：VS Code → 扩展面板 → ... → 从 VSIX 安装 → 选软件包内的 vuln-scanner-1.3.0.vsix；
- 2. 用法：打开代码文件 → 右键 → 「凿凿：分析当前文件」；结果面板除漏洞判定与修复建议外，还展示信任层明细（每条候选的置信度、裁决档位、共形预测、反事实验证、证据门），低置信候选单独列入“需人工复核”；每个确认的漏洞按行号标红色波浪线（可配置关闭）；
- 3. 批量：命令面板（Ctrl+Shift+P）→ 「批量扫描工作区」；
- 4. 常用设置（搜 vulnScanner）：后端地址 backendUrl、保存时自动扫描 autoScanOnSave（1.5s 防抖）、批量文件上限 workspaceMaxFiles。

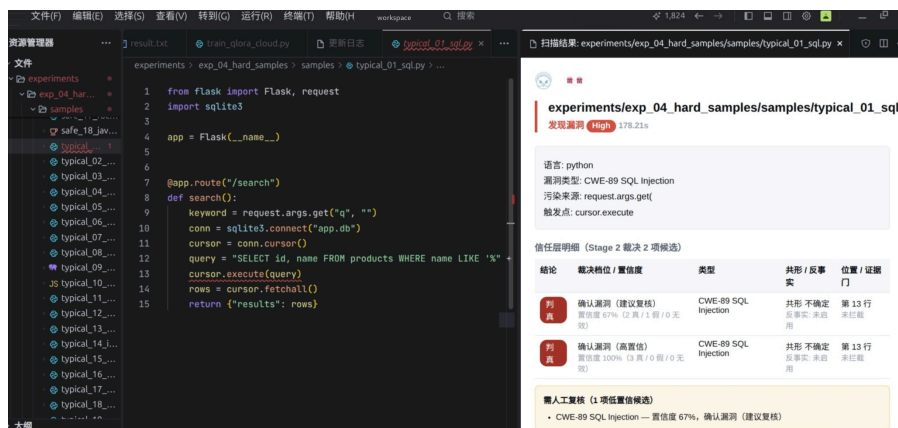


图 7-5 VS Code 插件：行级诊断与信任层明细（确认的漏洞按行号标红波浪线；右侧面板逐条候选展示裁决档位、置信度、CWE 类型与共形预测 / 反事实 / 证据门状态）

7.6.2 IntelliJ IDEA

1. 安装：File → Settings → Plugins → → Install Plugin from Disk... → 选 vuln-scanner-0.1.0.zip → 重启；
2. 用法：编辑器中选代码（不选则扫整个文件）→ 右键 → 喵喵 漏洞扫描（或 Ctrl+Alt+Shift+V），结果以气球通知弹出：结论、修复建议与信任层摘要（置信度·裁决档位·共形预测·反事实·证据门）。

IntelliJ 版定位为演示版：无波浪线标记与批量扫描。

7.7 命令行速查

无需打开网页，直接复用扫描引擎（不需要先启动后端服务）：

`python -m app.launcher.vuln_scanner_cli <命令>`

```
health                                # 健康检查（模型/服务是否可用）
scan app/main.py                      # 扫单个文件
scan app/main.py --format json        # 输出 JSON（方便脚本处理）
batch ./src --output report.md        # 批量扫目录 + 导出报告
batch ./src --limit 20 --no-recursive # 限量、不递归
url https://example.com              # 扫网页脚本
github <仓库地址>                    # 扫 GitHub 仓库（浅克隆）
```

通用参数：--model <模型名> 切模型、--rag 开知识库、--format text|json。

适合接进 CI 或脚本里做提交前检查（扫描失败时退出码为 1）。

7.8 离线使用与模型管理

- 完全离线运行：联网只是首次下载依赖和模型时需要。系统运行期不访问外网（HuggingFace 强制离线模式、Semgrep 规则已随软件本地化）；

- 换 / 加模型：Web 界面右上角模型管理抽屉可拉取、删除、切换模型（带实时下载进度）；
命令行方式：python -m app.launcher.vuln_scanner_cli --model <模型名> <命令>，或设环境变量 VULN_SCANNER_MODEL；



图 7-6 模型管理抽屉（当前活动模型、已安装模型、可拉取模型列表与「拉取」入口；下载带实时进度条）- 模型有两档精度（界面会显示当前形态）：- 发布模型 v9max（Ollama，约 4.7GB，兼容性最好）；- 高精度形态（Transformers + LoRA，检出质量对应实测指标）——启动器检测到自带模型且本机环境可运行时自动启用；跑不动会自动回退并打印说明；- 国内网络：依赖走国内镜像、模型走 HF 镜像，支持断点续传与断流自动换源，一般无需手动配置代理。

7.9 常见问题 FAQ

问题	处理
扫描很慢 (>60 秒/文件)	大概率在用 CPU 推理。看启动日志 [硬件检测]；有独卡却走 CPU 时重启启动脚本，或显式设 <code>VULN_SCANNER_BACKEND=ollama</code>
结果全是“需人工复核”	模型可能没正确加载： <code>python -m app.launcher.vuln_scanner_cli health</code> 检查；确认模型名一致
报显存不足 (OOM)	降低上下文窗口：设置环境变量 <code>VULN_SCANNER_NUM_CTX=4096</code>
启动报端口被占用	<code>netstat -ano findstr 8765</code> (Windows) / <code>lsof -i :8765</code> (Linux/macOS) 找到进程杀掉
VS Code 插件没反应	确认后端已启动且 <code>http://localhost:8765/api/health</code> 能访问；核对插件设置 <code>vulnScanner.backendUrl</code>
波浪线没显示	设置里 <code>vulnScanner.markDiagnostics</code> 需为 <code>true</code> ；个别无法定位到行的情况只展示面板结果
Linux 上模型没进项目目录	系统自带 ollama 服务抢占：执行一次 <code>sudo systemctl disable --now ollama</code> 再重启脚本
RAG 相关报错	多为知识库未初始化或旧向量库需重建（见第八篇故障排查）

7.10 已知边界（使用前请读一遍）

我们对能力边界持如实态度，以下是用户应该知道的：

1. 速度：语义分析的成本是时间——单文件典型 5~15 秒，长文件（自动切片后逐段分析）可能到分钟级。它定位是“审计增强”，不是秒级 lint；
2. “需人工复核”会占一定比例：系统对没有确定性证据的判定选择保守（宁可交给人工）。这是防漏报、防误判的设计代价，不是故障；
3. 两类漏洞是弱项：① 跨文件的复杂攻击路径（只扫单文件时看不到全局数据流）；② “缺失型”漏洞（认证缺失、CSRF、竞态等“该有而没有”的问题）——此类没有确定性校验手段，置信不足时都会转人工复核；
4. 两档部署形态精度不同：高精度形态（Transformers + LoRA）优于发布模型（Ollama 量化形态）。界面会如实显示当前用哪一档；
5. 结论是辅助，不是法律：所有“安全”判定都建议结合业务上下文理解——系统能看懂代码语义，但看不懂用户没写出来的需求约束。

第八篇 安装手册

本篇对应官方材料：安装手册

8.1 系统要求

条件	要求	说明
操作系统	Windows 10+ / Linux / macOS	三平台均支持
Python	3.10+	启动脚本会检测并自动安装依赖
pip	任意可用版本	首次运行自动安装依赖
磁盘空间	≥ 8 GB（建议 15 GB）	模型权重约 4.7 GB + 依赖包约 2 GB + 向量库缓存
内存	≥ 8 GB（建议 16 GB）	模型推理需要占用内存
GPU（可选）	NVIDIA / AMD(ROCm) / Apple Silicon ≥ 4 GB	无 GPU 时自动回退 CPU 推理（速度约为 GPU 的 1/10~1/20）
Ollama	首次运行自动安装	若自动安装失败，请手动从官网安装
git（可选）	任意版本	仅 GitHub 仓库扫描功能需要
网络	首次需要联网	下载依赖包 + 模型；后续可离线运行

Windows 下 LlamaCPP 后端暂不支持 RTX 50 系列与 AMD 显卡:受 Windows 上 llama-cpp-python 预编译二进制与编译工具链限制，这两类显卡易报 DLL / 编译类错误。推荐改走默认 Ollama 后端，或使用 Linux 系统运行 LlamaCPP / Transformers 后端。

8.2 一键启动（推荐）

一条命令搞定，启动脚本会自动完成其余所有事情：

```
# 解压软件包后进入项目根目录

# Windows: 双击
app\launcher\start_windows.bat

# Linux / macOS
bash app/launcher/start_linux_macos.sh
```

8.2.1 启动脚本自动完成的步骤

1. 检测并安装 **Python** 核心依赖——首次运行自动安装；
2. 选择推理后端——启动器会提示选择 Ollama / Transformers / LlamaCPP（Linux 另提供实验性 vLLM，Windows/macOS 隐藏该选项）；默认根据环境变量与硬件自动解析；

3. 自动安装后端专属依赖——若选择进程内后端，启动器会按当前 OS 与 GPU 自动下载正确的 `torch / transformers / peft / bitsandbytes / llama-cpp-python`（支持 Windows/Linux/macOS × NVIDIA/AMD/Apple/CPU）；
4. 检测并安装 **Ollama**——使用 Ollama 后端时自动安装/启动（其他后端跳过）；
5. 硬件检测与自适应配置——自动检测 GPU/CPU/RAM，按显存分档选择推理参数（`num_ctx / num_gpu / 量化等级`）；
6. 模型存储迁移与拉取——启动脚本已把模型存储环境变量锁定到项目目录（Ollama → `models/ollama`，Transformers 基座缓存 → `models/transformers/.hf_home`，`llama.cpp` → `models/llamacpp`）；首次启动还会把旧位置（如 `~/.ollama/models`）已有的 Ollama 模型剪切迁移进项目目录，不留系统盘。选择 Ollama 时自动拉取发布模型（约 4.7 GB，Q4 量化），拉取失败则回退官方 Qwen3-8B 基础模型；
7. 默认后端解析——项目 `models/` 下探测到自研 LoRA adapter 且当前运行时兼容时，默认采用 **Transformers** 进程内后端（基座 NF4 量化 + LoRA FP16，精度最高）；否则默认 Ollama（显存分档不满足时也会推荐回退）；
8. 召回传统安全工具——自动安装 Bandit / Semgrep / Gitleaks / Trivy / pip-audit / detect-secrets（两阶段 Stage 1 工具召回依赖；缺哪个静默跳过哪个）；
9. 启动后端——FastAPI 服务监听 `http://127.0.0.1:8765`；
10. 打开浏览器——自动跳转到 `http://localhost:8765`。

首次启动因需下载模型（约 4.7 GB）或后端依赖（torch 等约 2-4 GB），耗时取决于网速，请耐心等待。后续启动通常 10 秒内完成。

四种后端在三个平台上的模型存储都由启动器锁定在项目目录内，不会把系统盘 / 用户家目录塞满。

8.2.2 ⚠ Linux 用户：先停用系统级 Ollama 服务（只需一次）

四种后端的模型存储都由启动器锁定到项目目录；之所以只有 **Linux + Ollama** 需要手动处理一次，是因为 Ollama 是唯一以独立系统服务形态运行的后端——进程内后端（Transformers / LlamaCPP）与 vLLM 的存储路径直接由启动器注入的环境变量决定，不存在“别的服务抢着管存储”的问题；而 Linux 上用官方脚本自动安装的 Ollama 会注册成 **systemd** 服务：开机自启、以系统默认存储（`/usr/share/ollama`）占用 11434 端口，且该服务属于系统用户——启动器无权限停止它（`kill` 杀掉后 **systemd** 还会自动重启）。首次使用前请执行一次：

```
sudo systemctl disable --now ollama
```

之后启动器会以项目目录为存储启动自己的 `ollama serve`，模型全部落在 `models/ollama`（启动器也会先尝试自动停用外部服务，失败时打印同一条命令；预先执行一次可免去现场交互）。

Windows / macOS 无需此步骤：winget / brew 安装不注册系统服务，启动器直接接管即可（Windows 版 Ollama 桌面程序被停止后会自动重启并抢占端口，启动器已内置结束托盘程序 + 重试接管的逻辑）。

8.3 手动启动

如果不想用启动脚本（例如已在 IDE 中配置好环境），可以手动分步启动：

```
# 1. 确保 Ollama 运行
ollama serve &

# 2. 确保模型已拉取
ollama pull <发布模型名>

# 3. 启动后端
uvicorn app.backend.main:app --host 127.0.0.1 --port 8765

# 4. 浏览器访问
open http://localhost:8765      # macOS
xdg-open http://localhost:8765  # Linux
start http://localhost:8765     # Windows
```

手动方式绕过了启动器：此时 `ollama serve / ollama pull` 使用 Ollama 默认存储（`~/.ollama`），不会自动落到项目目录。如需与一键启动一致的存储位置，先执行 `export OLLAMA_MODELS=<项目>/models/ollama`（Windows：set OLLAMA_MODELS=...）再 `serve / pull`。

8.4 确认安装成功

浏览器打开 `http://localhost:8765`，看到欢迎页 / 仪表盘即成功。也可在终端验证：

```
python -m app.launcher.vuln_scanner_cli health
```

8.5 切换推理后端

后端按环境变量显式设置 → **adapter** 探测 → **Ollama** 兜底的优先级自动解析，完整规则与设计依据见第三篇 §3.5.2。常用切换方式：

```
# Transformers 后端 (NF4 基座 + FP16 LoRA, 需 6GB+ 显存或足够内存)
export VULN_SCANNER_BACKEND=transformers
export VULN_SCANNER_ADAPTER=/path/to/alpha05_stage2_lora
bash app/launcher/start_linux_macos.sh

# LlamaCPP 后端 (Q4 GGUF + 运行时 FP16 LoRA, 实验性)
export VULN_SCANNER_BACKEND=llamacpp
export VULN_SCANNER_GGUF=/path/to/q4_k_m.gguf
export VULN_SCANNER_ADAPTER=/path/to/alpha05_stage2_lora
bash app/launcher/start_linux_macos.sh
```

Windows 使用 `set` 代替 `export`。选择进程内后端后，启动器会自动按当前 OS/GPU 下载对应版本的依赖。

8.5.1 后端平台支持矩阵

各推理后端在不同平台 / 显卡上的支持情况，完整矩阵见第三篇 §3.5.3（含 vLLM 行），此处给出安装选型结论：

结论	说明
优先 Ollama	三平台全兼容 (Windows / Linux / macOS × NVIDIA / RTX 50 系 / AMD / Apple / CPU)，依赖最少，是设备兼容性优先时的首选
Transformers 限 Linux 体验最佳	Windows 端不支持 AMD，RTX 50 系需额外 CUDA Toolkit
LlamaCPP 的 Windows 端受限	暂不支持 RTX 50 系与 AMD 显卡 (源码编译未成功)
受限组合的处理	改用 Linux 运行，或回退 Ollama 后端

完整能力矩阵 (含 Apple Silicon 与纯 CPU 列) 以第三篇 §3.5.3 为准，两处如有出入以第三篇为唯一事实来源。

8.5.2 显存与平台说明

- Transformers 后端推荐 NVIDIA/AMD 显存 $\geq 6\text{GB}$ ；4GB 显存会被强制走 CPU，速度显著下降；
- bitsandbytes 已支持 NVIDIA CUDA、AMD ROCm (预览)、CPU-only、Apple Silicon (慢速路径)；追求速度请改用 Ollama 后端；
- LlamaCPP 后端可纯 CPU 运行，也可按编译选项启用 CUDA/ROCm/Metal。

8.6 环境变量配置

启动器和后端通过以下环境变量控制行为，按需设置：

环境变量	默认值	说明
VULN_SCANNER_BACKEND	自动解析	推理后端：ollama / transformers / llamacpp
VULN_SCANNER_ADAPTER	自动探测	Transformers/LlamaCPP 后端：LoRA adapter 目录
VULN_SCANNER_GGUF	无	LlamaCPP 后端必填：Q4 GGUF 基座文件路径
VULN_SCANNER_MODEL	发布模型	Ollama 后端使用的模型名
VULN_SCANNER_FALLBACK_MODEL	基础模型	主模型拉取失败时的回退模型
VULN_SCANNER_AUTO_INSTALL_DEPS	未设置	¹ 强制重新检查/升级依赖； ⁰ 禁用自动安装
VULN_SCANNER_PIP_INDEX	无	覆盖 pip 镜像源
VULN_SCANNER_RAG	0	设为 ¹ 启用知识库增强
VULN_SCANNER_PREFILTER	1	设为 ⁰ 关闭传统工具预筛

环境变量	默认值	说明
OLLAMA_BASE_URL	http://localhost:11434	Ollama 服务地址
VULN_SCANNER_NUM_CTX	自动检测	模型上下文窗口大小
VULN_SCANNER_NUM_GPU	自动检测	GPU 层数 (0 = 纯 CPU)
VULN_SCANNER_NUM_THREAD	自动检测	CPU 推理线程数

8.7 卸载

8.7.1 一键卸载

平台	入口
Windows	双击 <code>uninstall_windows.bat</code> ，或运行 <code>python uninstall.py</code>
Linux / macOS	运行 <code>bash uninstall.sh</code> ，或运行 <code>python3 uninstall.py</code>

常用参数：

- `--yes`：全自动，跳过所有确认；
- `--dry-run`：模拟，只列出将要删除的内容，不实际删除；
- `--keep-project`：保留项目文件夹（只清依赖和数据）；
- `--keep-ollama`：保留 Ollama 本体与模型；
- `--keep-accel`：保留 CUDA/ROCm 系统组件。

8.7.2 ⚠ 必须用安装时的同一个 Python 环境卸载

依赖装进哪个 Python 环境，就必须用哪个环境卸载：

- 一键启动用户：依赖是启动器用系统 Python 自动安装的，直接运行卸载脚本即可；
- **conda / venv** 用户：安装时先激活过环境，卸载前也必须先激活同一个环境再运行卸载脚本；
- 卸载脚本只清理“运行它的那个 Python 环境”，换环境运行会提示“未发现本项目相关包”并跳过，等于没有真正卸载。

8.7.3 卸载范围

脚本会按本机实际检测结果动态清理：

1. 后端进程（端口 8765）与 Ollama 服务进程；
2. 当前 Python 环境中的本项目相关包与 pip 下载缓存；
3. Ollama 拉取的模型；
4. Ollama 本体（按平台：winget / brew / apt / dnf / pacman / zypper / apk / 官方脚本）；
5. Linux 系统级 CUDA / ROCm 组件（需 sudo，仅 apt 系发行版）；
6. 本地运行数据：向量库、输出目录、日志、各类缓存；
7. 已安装的编辑器插件（尽力而为）；
8. 整个项目文件夹。

8.7.4 不会自动删除的内容

- 外部扫描工具 (bandit / semgrep / gitleaks / trivy)：这些是共享系统工具，可能被其他项目使用，卸载脚本不会动它们；
- **conda** 环境本身：卸载脚本只清当前环境里的包，不会删除任何 conda 环境；
- **npm** 全局包：如需删除请手动操作；
- 部分共享基础库（网络/类型相关的基础库）：脚本保留这类通用库，避免影响同一环境里的其他程序。但注意：科学计算类包也在清理列表里，若同一环境还有其他项目用到它们，请留意确认提示，必要时先用 `--dry-run` 查看或将依赖装在独立虚拟环境。

8.7.5 注意事项

- Ollama 模型目录与 HuggingFace 缓存是全局目录，卸载会删除里面所有模型/缓存（包括其他项目用到的），`--yes` 模式下不会再次确认，请提前确认；
- 卸载脚本会逐项检查实际删除结果，失败项会明确报错，不会提示虚假成功；完成后请确认项目文件夹确实已删除。

8.8 故障排查

8.8.1 Semgrep 规则本地化（离线可用，随软件分发）

Semgrep 规则包已本地化并随软件入库（版本固定），扫描完全离线运行；规则的更新 / 校验命令与缺失降级行为见第五篇 §5.2.4。

8.8.2 启动相关

问题	原因	解决方案
<code>ModuleNotFoundError: No module named 'app.launcher.bootstrap'</code>	未在项目根目录下运行	确保 <code>cd</code> 到项目根目录后再执行启动脚本；Windows 启动脚本已内置自动切到根目录
<code>pip install</code> 找不到依赖清单文件	当前目录不是项目根目录	<code>cd</code> 到项目根目录后再安装
<code>ModuleNotFoundError: No module named 'fastapi'</code> （或 <code>chromadb</code> / <code>tree_sitter</code> 等）	依赖未安装	重新运行启动脚本（会自动安装），或手动安装依赖
启动器提示 Ollama 自动安装失败	包管理器不可用或网络问题	手动从 Ollama 官网下载安装，安装后重启终端再运行启动脚本
启动器提示 Ollama 已安装但不在 PATH 中	安装后 PATH 未刷新	重启终端；或手动将 Ollama 安装路径加入系统 PATH
后端启动超时	端口 8765 被占用	检查端口占用并杀掉占用进程后重试

问题	原因	解决方案
Linux 上模型没统一到项目目录	系统级 ollama 服务仍用系统存储	执行一次 <code>sudo systemctl disable --now ollama</code> ，再由启动器接管

8.8.3 模型相关

问题	原因	解决方案
模型下载超时或失败	网络不稳定或模型源不可达	手动重试拉取；若持续失败可改用回退模型
扫描结果全为“无法判定”	模型未正确加载或输出格式不匹配	运行健康检查确认模型可用性；确认模型名与已安装的一致
推理速度极慢 (> 60s/文件)	无 GPU 回退到 CPU 推理	检查启动日志中的硬件检测行；CPU 模式约为 GPU 的 1/10 速度，属正常现象
OOM (显存溢出)	上下文窗口过大或显存不足	降低上下文窗口：设置环境变量 <code>VULN_SCANNER_NUM_CTX=4096</code>
进程内后端报 DLL / 编译类错误	显卡架构缺少对应运行时	在 Linux 上运行，或改用默认 Ollama 后端

8.8.4 RAG / 向量库相关

问题	原因	解决方案
ModuleNotFoundError: No module named 'chromadb'	未安装 RAG 依赖	安装依赖清单中的 chromadb
RAG 扫描报错 <code>embedding model not found</code>	embedding 模型未缓存到本地	在有网络的环境执行一次模型下载（国内可用镜像）
RAG 检索结果明显不对 / 向量库为空	向量库未构建或由旧 embedding 模型构建	删除旧向量库目录后重建
自定义 embedding 缓存路径	默认缓存路径不可写	设置 <code>CHROMA_EMBEDDING_MODEL_PATH=/path/to/local/model</code>

8.8.5 插件相关

问题	原因	解决方案
VS Code 插件扫描无响应	后端未启动或地址不匹配	确认 <code>http://localhost:8765/api/health</code> 可访问；检查插件设置中的后端地址
VS Code 插件请求超时	大文件推理耗时长	在设置中将请求超时调大（默认 300000ms = 5 分钟）
IntelliJ 插件编译失败	JDK 版本低于 17 或构建同步失败	确认 JDK 17+；在 IDEA 中重新同步构建
IntelliJ 插件通知不弹出	后端地址不匹配	修改插件源码中的后端地址常量后重新构建

8.8.6 IDE 报红（不影响运行）

问题	原因	解决方案
IDE 中大量红色错误标记	Python 解释器选错、依赖未安装、模块路径未配置	1. 选择正确的 Python 解释器；2. 在项目根目录安装项目包；3. 在 IDE 配置中把项目根目录加入分析路径
类型检查器报类型不匹配	严格模式误报	不影响运行，可调整类型检查严格度

8.9 目录结构

软件根目录/

```
|— app/                                # 软件本体
|   |— backend/                        # FastAPI 后端 (REST API + 两阶段扫描引擎)
|   |— launcher/                      # 跨平台启动器 (一键启动 + 依赖自动安装)
|   |— vscode-extension/              # VS Code 插件源码
|   |— intellij-extension/            # IntelliJ 插件源码
|— releases/                          # 插件安装包 (vsix / zip)
|— models/
|   |— semgrep_rules/                 # 本地化 Semgrep 规则 (离线可用)
|— graduation_project/                 # 核心 Python 包 (扫描引擎 / schema / prompts)
|— data/                              # 本地运行数据 (向量库等)
|— docs/                              # 用户手册等文档
|— requirements.txt
|— uninstall.py / uninstall_windows.bat / uninstall.sh
|— README.md
```

附录

附录 A 环境要求表

系统运行的软硬件环境要求见第八篇 §8.1。

附录 B CLI 命令速查

命令	功能	示例
health	健康检查（服务 + 模型可用性）	python -m app.launcher.vuln_scanner_cli health
scan <file>	扫描单个文件	python -m app.launcher.vuln_scanner_cli scan app/main.py
batch <dir>	批量扫描目录下所有代码文件	python -m app.launcher.vuln_scanner_cli batch ./src --output report.md
url <url>	扫描 URL 抓取的脚本	python -m app.launcher.vuln_scanner_cli url https://example.com
github <repo>	扫描 GitHub 仓库（浅克隆）	python -m app.launcher.vuln_scanner_cli github <仓库地址>

通用参数（所有子命令可用）：

参数	默认值	说明
--model	发布模型	切换模型
--ollama-url	http://localhost:11434	Ollama 服务地址
--rag	关闭	启用知识库增强（较慢但更准）
--format	text	输出格式：text（终端着色）或 json（结构化）

子命令专属参数：

子命令	参数	默认值	说明
scan	--language	自动识别	覆盖语言检测（如 python / javascript）
scan / batch / url / github	--output	无	报告输出路径（.md 或 .json）
batch	--limit	0（不限）	最多扫描文件数
batch	--no-recursive	关闭	不递归子目录
github	--max-files	50	最多扫描文件数
scan / batch	--verbose	关闭	显示详细分析

退出码契约：健康检查返回 0/1；扫描有错误返回 1；用户中断（Ctrl+C）返回 130；其他异常返回 1。

支持的文件类型：.py .js .ts .jsx .tsx .java .php .go .html .htm .vue .svelte

附录 C 输入限制与支持范围

项目	限制
代码语言	Python / JavaScript / TypeScript / Java / PHP / Go / HTML
文件类型	.py .js .ts .jsx .tsx .java .php .go .html .htm .vue .svelte
单段代码	最多 200 万字符
单文件	最大 2 MB
单次批量	最多 200 个文件、总计 10 MB
GitHub 扫描	浅克隆 + 120 秒超时 + 文件数上限（网页端默认 20，接口默认 50，可调，上限 500；由 MAX_SCAN_FILES_CEILING 与收集上限绑定）
URL 抓取	响应体流式读取，超过 2 MB 截断

附录 D 评估口径说明

本篇口径为项目统一评估口径（2026-09-08），全部数字可由随附评估脚本复现。

D.1 指标定义

指标	定义
acc	$(TP + TN) / (TP + TN + FP + FN)$ （仅在已裁决样本上计算）
recall	漏洞样本被正确判出的比例（实例级）
FPR	安全样本被误判为漏洞的比例
strict	判真且 CWE 归因正确（经归一化 + 父子族匹配）
micro / macro	实例级 recall 的两种平均方式
加权	风险加权分（冻结权重表）
未决率	系统不做自动结论、转人工复核的样本占比（与 acc 成对引用）
parse_fail	模型输出无法解析的比例（计入漏报，不剔除出分母）

D.2 口径须知

- 一套分数规则走全程——所有表格使用同一套定义，数字可由评估脚本复现；
- 两阶段系统——未决率与 acc 成对引用；“干净评估”（关闭信号回填）与“完整信任层”（开启回填）引用时须注明组态；
- 推理管道——高精度形态（NF4 + FP16 LoRA）与发布形态（Q4_K_M 合并量化）指标不同，引用须注明管道；
- 全正样本集——只有 recall / acc 有意义，FPR 无定义（记「-」）。

D.3 测试面构成

测试面	规模	构成	典型性
合成集	87 段	合成教学样本	72%
真实 CVE 片段	20 段	真实 CVE 最小化片段	60%
真实完整文件	50 段	真实完整文件（哈希冻结）	6%

D.4 术语口径提醒

- 「召回率」统一写「recall」，避免中英混用；
- 「CWE」编号写法统一为 `CWE-数字`（带连字符）；
- 日期统一 ISO 格式；
- 未实测的估计值不得写入（如“多少比例的文件不进 LLM 裁决”一类推断）。